UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Arquitetura de Monitoramento e Observabilidade de Aplicações em Kubernetes

Cauan Marcelo Lima

Orientador: Prof. Dr. Carlos Manuel Dias Viegas

Natal, RN, 10 de dezembro de 2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Arquitetura de Monitoramento e Observabilidade de Aplicações em Kubernetes

Cauan Marcelo Lima

Orientador: Prof. Dr. Carlos Manuel Dias Viegas

Trabalho de Conclusão de Curso de Graduação na modalidade Monografia, submetido como parte dos requisitos necessários para conclusão do curso de Engenharia de Computação pela Universidade Federal do Rio Grande do Norte (UFRN/CT).

Natal, RN, 10 de dezembro de 2025

Universidade Federal do Rio Grande do Norte - UFRN

Sistema de Bibliotecas - SISBI

Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Lima, Cauan Marcelo.
Arquitetura de monitoramento e observabilidade de aplicações em kubernetes / Cauan Marcelo Lima. - 2025.
81 f.: il.

Monografia (graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Curso de Engenharia de Computação.
Orientação: Prof. Dr. Carlos Manuel Dias Viegas

1. Monitoramento - Monografia. 2. Observabilidade - Monografia. 3. Kubernetes - Monografia. I. Viegas, Carlos Manuel Dias. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Jackeline dos Santos Pinheiro da Silva Maia Cavalcanti - CRB-15/317

Arquitetura de Monitoramento e Observabilidade de Aplicações em Kubernetes

Cauan Marcelo Lima

pelos seguintes membros:

Prof. Dr. Carlos Manuel Dias Viegas (orientador) . . . . . DCA/UFRN

Prof. Dr. Eduardo de Lucena Falcão . . . . . DCA/UFFN

Prof. Dr. Ivanovitch Medeiros Dantas da Silva . . . . . DCA/UFRN

Deus, à minha mãe e ao meu pai.

Agradecimentos

Ao meu orientador, Carlos Manuel Dias Viegas.

Aos professores Darlan Araújo Moreira, Eduardo de Lucena Falcão, Heitor Medeiros Florencio, Ivanovitch Medeiros Dantas da Silva e Luiz Affonso Henderson Guedes de Oliveira.

Ao servidor do bloco de aulas do curso de Pedagogia, Sr. Sérgio, com quem atuei como bolsista de apoio técnico.

Aos meus amigos Frederico Pranto, Werbet Luiz, Paula Souza e Cintia Nunes.

À minha família pelo apoio durante esta jornada.

Ao meu pai e à minha mãe.

À minha avó do coração.

Aos servidores e funcionários da UFRN.

Aos demais colegas de graduação, pelas críticas e sugestões.

À CAPES/CNPQ e Funpec, pelo apoio financeiro.

A Deus.

Resumo

Em um cenário de crescente transformação digital, a alta disponibilidade, resiliência e escalabilidade de serviços tornaram-se exigências do mercado, pois impactam diretamente a experiência dos usuários. Simultaneamente, o Kubernetes vem se consolidando como uma plataforma de hospedagem que oferece diversos benefícios, contudo, por si só, não detém um conjunto completo de recursos de monitoramento e observabilidade. Este estudo apresenta uma alternativa baseada em ferramentas *open-source* capazes de oferecer um ambiente que proporcione diagnóstico e colabore com o aperfeiçoamento de serviços hospedados. A arquitetura proposta fornece um modelo eficiente, acessível e robusto para monitoramento e observabilidade de aplicações executadas em *containers* orquestrados por Kubernetes. Integrando componentes como Argo CD, HashiCorp Vault para a gestão segura de *secrets* e um conjunto de ferramentas de observabilidade composto por OpenTelemetry, Prometheus, Grafana Tempo, Coroot e Netdata, a abordagem foi validada experimentalmente por meio de cenários envolvendo execução de serviços utilizando banco de dados, testes de sobrecarga de CPU, memória e disco e simulação de sistema com falhas. Os resultados demonstram a capacidade da arquitetura em identificar gargalos, detectar incidentes, monitorar o estado dos serviços e registrar anomalias do *cluster* e dos sistemas hospedados. Conclui-se que é plenamente viável construir um ambiente de observabilidade moderno e confiável utilizando tecnologias que visam garantir segurança, rastreabilidade e diagnósticos, estabelecendo uma base sólida para ambientes Kubernetes alinhados às demandas contemporâneas de operação.

Palavras-chave: Monitoramento; Observabilidade; Kubernetes.

Abstract

In a scenario of increasing digital transformation, high availability, resilience, and service scalability have become market requirements as they directly impact user experience. Concurrently, Kubernetes has emerged as a hosting platform that offers numerous benefits, yet by itself it does not provide a complete set of monitoring and observability features. This study presents an alternative based on open-source tools capable of delivering an environment that enables diagnostics and supports the improvement of hosted services. The proposed architecture provides an efficient, accessible, and robust model for monitoring and observing applications running in containers orchestrated by Kubernetes. By integrating components such as Argo CD, HashiCorp Vault for secure secrets management, and an observability toolset composed of OpenTelemetry, Prometheus, Grafana Tempo, Coroot, and Netdata, the approach was experimentally validated through scenarios involving service execution with databases, CPU, memory, and disk stress tests, and the simulation of systems with failures. The results demonstrate the architecture's ability to identify bottlenecks, detect incidents, monitor service health, and record anomalies within the cluster and hosted systems. It is concluded that it is fully feasible to build a modern and reliable observability environment using technologies that ensure security, traceability, and diagnostics, establishing a solid foundation for Kubernetes environments aligned with contemporary operational demands.

Keywords: Monitoring; Observability, Kubernetes.

Sumário

Sumário i

Lista de Figuras iii

Lista de Tabelas v

Lista de Siglas vii

1 Introdução 1

1.1 Objetivos Gerais 2

1.2 Objetivos Específicos 3

1.3 Estrutura do Trabalho 3

2 Fundamentação Teórica 5

2.1 Observabilidade e Monitoramento 5

2.2 Kubernetes: Orquestração de *containers* 6

2.3 Helm: Gerenciamento de Aplicações no Kubernetes 7

2.3.1 Repositórios de Helm Charts 7

2.4 Registro de Imagens de *Containers* 8

2.5 Argo CD 9

2.5.1 *Backup* e Recuperação de Desastres no Argo CD 10

2.6 OpenTelemetry 10

2.6.1 Application Performance Management 12

2.7 Coroot 13

2.8 Netdata 14
2.9 Netdata vs Coroot 15
2.10 Prometheus 16
2.11 Grafana 17
2.12 Grafana Tempo 18
2.13 HashiCorp Vault 18

3 Desenvolvimento **21**
3.1 Implementação do sistema 21
3.2 Arquitetura do sistema 22

4 Experimentos e Resultados **27**
4.1 Caso 1 - Calculadora com histórico em banco de dados 27
4.2 Caso 2 - Testes de sobrecarga 32
4.3 Caso 3 - Api com alto índice de erro 35
4.4 Visão Geral 37
4.4.1 Coroot 37
4.4.2 Netdata 43
4.4.3 Prometheus 44

5 Conclusão **47**
5.1 Trabalhos Futuros 48

Referências bibliográficas **50**

A Informações adicionais **53**

Lista de Figuras

3.1 Sistemas instalados a partir do Argo CD 22

3.2 IDE do Argo CD 22

3.3 Fluxo de dados entre os sistemas 25

3.4 *Containers* que compõem um *pod* 25

4.1 Visão geral do sistema Calculadora 28

4.2 Gráfico Open Telemetry APM 30

4.3 Grafana Tempo no Grafana 30

4.4 Inspeção de trace 31

4.5 Mapa do fluxo de rede 32

4.6 Listagem de *logs* 32

4.7 Tela sobre memória RAM 33

4.8 Tela sobre CPU 34

4.9 Visão geral de alertas na IDE do Netdata 34

4.10 Alerta de disco do Netdata no Discord 35

4.11 Tracing via eBPF 36

4.12 Incidente relacionado a API 37

4.13 Notificação de incidente no Discord 37

4.14 Lista de sistemas 38

4.15 Incidente relacionado ao mongoDB 39

4.16 Incidente com MongoDB 39

4.17 Tela de MongoDB 40

4.18 Tráfego de rede de sistema 40

4.19 Mapa de serviços 41
4.20 Registro de alta latência 41
4.21 Tela de estimativa de custos 42
4.22 Tela de riscos 42
4.23 Profiling do MongoDB 43
4.24 Tela de Inspeção de sistemas no Netdata UI 44
4.25 Dashboard Prometheus sobre uso memória RAM dos pods 45
4.26 Dashboard Prometheus sobre uso de CPU dos pods 45

A.1 Manifesto do sistema dotnet-k8s-math 58
A.2 Manifesto do serviço Vault 59

Lista de Tabelas

A.1 Utilidade das ferramentas 54
A.2 Principais incidentes detectados pelo Coroot 55
A.3 Dashboards do Grafana utilizados para monitoramento 56
A.4 Serviços Instalados no Cluster via Helm Charts 56
A.5 Comparativo entre Coroot e Netdata 57

Lista de Siglas

API	*Application Programming Interface*
AI	*Artificial Intelligence*
CLI	*Command Line Interface*
CD	*Continuous Delivery*
CI	*Continuous Integration*
CRD	*Custom Resource Definition*
eBPF	*extended Berkeley Packet Filter*
IDE	*Integrated Development Environment*
UI	*User Interface*
APM	*Application Performance Management*
CPU	*Central Processing Unit*
IaC	*Infrastructure as Code*
HTTP	*Hypertext Transfer Protocol*
TI	Tecnologia da Informação
VM	*Virtual Machine*
IoT	*Internet of Things*
ACL	*Access Control List*
LDAP	*Lightweight Directory Access Protocol*
gRPC	*Google Remote Procedure Call*
OTLP	*OpenTelemetry Protocol*
RBAC	*Role-Based Access Control*
SSO	*Single Sign-On*
SCIM	*System for Cross-domain Identity Management*
YAML	*YAML Ain't Markup Language*
JSON	*JavaScript Object Notation*
RED	*Rate, Errors and Duration*
RAM	*Random Access Memory*
SLO	*Service Level Objective*
OOM Kill	*Out Of Memory Kill*

Capítulo 1

Introdução

Em meio à crescente tendência de transformação digital, organizações de diferentes portes e setores têm buscado soluções tecnológicas capazes de elevar a disponibilidade, otimizar o desempenho e aumentar a agilidade na entrega de serviços. Nesse contexto, alta disponibilidade, resiliência e escalabilidade passaram a se consolidar como requisitos essenciais para garantir a continuidade das operações em um mercado dinâmico e altamente exigente.

Para compreender as tendências que impulsionam esse movimento, o relatório "The Voice of Kubernetes Experts Report 2024"da Portworx em parceria com a Dimensional Research, entrevistou 527 profissionais qualificados diretamente responsáveis pela execução ou gerenciamento de serviços de dados em ambientes Kubernetes, em empresas com 500 ou mais funcionários. Os participantes representam um público altamente especializado, sendo que 91% possuem ao menos dois anos de experiência com Kubernetes e 56% têm mais de quatro anos de atuação. As funções exercidas abrangem diferentes segmentos organizacionais, incluindo 39% de diretores e gerentes, 21% de profissionais de engenharia de plataforma, 20% de executivos e 19% de especialistas em banco de dados. Os dados estatísticos revelam uma tendência clara de adoção: 58% das organizações planejam migrar parte de suas máquinas virtuais para o gerenciamento via Kubernetes, e a maioria, 65%, pretende concluir essa transição nos próximos dois anos. A motivação por trás dessa mudança está associada à busca por maior escalabilidade, flexibilidade,

simplicidade operacional e redução de custos (PORTWORX, 2024).

O avanço das práticas modernas de computação impulsionou o surgimento de plataformas de hospedagem e gerenciamento de serviços baseadas em Kubernetes. No entanto, adotar Kubernetes não garante, por si só, visibilidade adequada do ambiente. É indispensável associar ao *cluster* um conjunto de ferramentas capazes de fornecer dados analisáveis, métricas detalhadas e monitoramento contínuo, assegurando a detecção rápida de falhas e a otimização constante do desempenho.

Monitoramento e observabilidade tornam-se elementos centrais nesse processo. O monitoramento permite acompanhar o comportamento das aplicações em tempo real, enquanto a observabilidade possibilita uma análise do ecossistema, correlacionando métricas, *logs* e *traces* para identificar anomalias e acelerar a resolução de incidentes. A capacidade de observabilidade de sistemas permite identificar os gargalos do serviço, evidenciando onde ocorrem as maiores latências na entrega. Ao localizar esses pontos de falha, torna-se possível direcionar recursos, seja em aprimoramento ou computacional, de forma mais precisa, com o intuito de melhorar a experiência do usuário final.

Entretanto, muitas organizações não dispõem de recursos para contratar plataformas terceirizadas ou serviços em nuvem. Surge, então, a motivação deste trabalho: propor uma arquitetura de monitoramento e observabilidade acessível em Kubernetes, robusta e confiável, construída com tecnologias modernas e amplamente utilizadas no mercado para empresas que desejam modernizar sua infraestrutura ou iniciar uma arquitetura de sistemas em Kubernetes, demonstrando que é possível estruturar um ambiente eficiente sem depender de soluções proprietárias e sem altos investimentos.

1.1 Objetivos Gerais

Propor uma arquitetura em Kubernetes baseada em boas práticas e em ferramentas de monitoramento e observabilidade, demonstrando a importância desses três elementos

para a operação eficiente de sistemas modernos.

1.2 Objetivos Específicos

Propor uma arquitetura de software em Kubernetes, isto é, arquitetar um conjunto de ferramentas que, integradas ao Kubernetes, promovam monitoramento e observabilidade. Utilizar solução para a implantação das ferramentas e serviços com práticas de GitOps e propor uma ferramenta de gestão de *secrets*.

Prover monitoramento e observabilidade utilizando Coroot, Netdata, Prometheus, OpenTelemetry e Grafana Tempo, destacando o diferencial de cada ferramenta. Evidenciar a importância de cada informação que é possível obter dos sistemas hospedados e como cada ferramenta contribui para a infraestrutura. E demonstrar como o HashiCorp Vault se integra à plataforma, trazendo gerenciamento seguro de *secrets*.

1.3 Estrutura do Trabalho

A estrutura deste trabalho está organizada em cinco capítulos, além das seções complementares. O Capítulo 1 apresenta a introdução, contextualizando o tema, os objetivos e a relevância da proposta. O Capítulo 2, denominado Fundamentação Teórica, aborda os conceitos e as tecnologias que sustentam o desenvolvimento da plataforma, incluindo observabilidade em sistemas distribuídos, orquestração de *containers* em Kubernetes e as ferramentas empregadas, como Helm, Harbor, ArgoCD, OpenTelemetry, Coroot, Netdata, Prometheus, Grafana, Grafana Tempo e HashiCorp Vault. O Capítulo 3 descreve o desenvolvimento da solução, detalhando a arquitetura do sistema e as integrações realizadas. O Capítulo 4 apresenta os experimentos e resultados, demonstrando o funcionamento dos componentes da plataforma nas áreas de segurança, observabilidade e monitoramento. Por fim, o Capítulo 5 apresenta a conclusão, sintetizando os resultados alcançados e indi-

cando possíveis aprimoramentos para trabalhos subsequentes.

Capítulo 2

Fundamentação Teórica

2.1 Observabilidade e Monitoramento

Com o crescimento das arquiteturas orientadas a microsserviços e de plataformas como Kubernetes é crucial ter visibilidade sobre o comportamento dos sistemas. A noção de observabilidade se destaca neste novo mundo como uma continuação das práticas de monitoramento anteriores.

Em consonância com (MAJORS; FONG-JONES; KUELBS, 2022), observa-se que a observabilidade vai além da simples coleta de dados. Trata-se da capacidade de compreender o comportamento do sistema, inclusive diante de questões complexas, mesmo sem conhecimento prévio do problema. Mais do que armazenar informações e gerar gráficos de métricas, a observabilidade consiste em possibilitar a formulação de perguntas *ad-hoc* sem a necessidade de inserir novo código. Nesse contexto, os *logs* fornecem detalhes sobre eventos ocorridos no sistema, as métricas evidenciam tendências operacionais e os *traces* indicam, de forma precisa, onde se encontram os gargalos de desempenho.

Simultaneamente, o monitoramento é o processo contínuo de analisar e interpretar informações sobre o funcionamento de um sistema, serviço, processo ou ambiente, com a finalidade de identificar problemas de forma rápida e gerar alertas que apoiem a tomada de decisões de maneira mais ágil possível.

2.2 Kubernetes: Orquestração de *containers*

Segundo (KUBERNETES, 2025), Kubernetes é uma plataforma de orquestração de *containers* de código aberto, inicialmente desenvolvida pelo Google e atualmente mantida pela Cloud Native Computing Foundation, que automatiza o gerenciamento, a implantação e a escalabilidade de aplicações em *containers*[1].

Com o aumento do uso de *containers*, especialmente com ferramentas como o Docker, surgiu a necessidade de uma forma simples e programática de agendar múltiplos *containers* em múltiplas máquinas. O Kubernetes atende a essa demanda ao introduzir abstrações de alto nível como *Pods, Services, Deployments, Namespaces*, entre outras, facilitando o controle de aplicações complexas. Esses recursos são implementados por meio de manifestos, que podem ser arquivos tipo YAML (YAML Ain't Markup Language) ou JSON (JavaScript Object Notation), nos quais se define o estado desejado dos objetos dentro do *cluster*.

O Kubernetes apresenta diversos pontos principais que o tornam uma plataforma robusta para a orquestração de *containers*. Ele realiza escalamento, ajusta o número de instâncias de aplicação conforme a carga de trabalho, podendo inclusive aumentar os recursos computacionais da mesma [2]. Provê resiliência ao realocar *containers* em caso de falha de um *node*[3]. Também permite atualizações seguras, isto é, *deploy* de novas versões passa por processos de validação antes de ficarem operante, podendo ser realizados por meio de procedimentos sofisticados geridos por ferramentas como Argo Rollouts[4] que proporcionam uma atualização gradativa, com testes em cada etapa.

[1] Um *container* é uma unidade leve e portátil de software que empacota o código de uma aplicação junto com suas dependências, bibliotecas e configurações necessárias para a execução em qualquer ambiente computacional de forma isolada.

[2] Para o aumento automático de recursos computacionais, é preciso implementar a ferramenta Vertical Pod Autoscaler (VPA)

[3] É uma máquina (física ou virtual) que faz parte do *cluster* e executa os *pods*, que contêm os *containers*.

[4] Argo Rollouts é um controlador para Kubernetes que implementa estratégias avançadas de implantação, permitindo atualizações progressivas.

2.3 Helm: Gerenciamento de Aplicações no Kubernetes

O Helm é o gerenciador de pacotes do Kubernetes, responsável por simplificar a implantação e o gerenciamento de aplicações dentro do *cluster* por meio de *charts* que definem os recursos necessários (HELM, 2025).

Com o Helm, as aplicações são empacotadas em *charts*, que são coleções de manifestos que descrevem os componentes necessários para a implantação de um sistema, podendo conter *Deployments*, *Services*, *ConfigMaps*, *Secrets*, entre outros. Esses *charts* podem ser versionados e compartilhados em repositórios, o que garante padronização e escalabilidade nas implantações.

Utilizando o Helm, é possível instalar serviços no *cluster* por meio dos *charts* que possuem parâmetros configuráveis. Esses parâmetros são variáveis que permitem customizar recursos como CPU (Central Processing Unit) e memória, portas, réplicas de *pods*, credenciais e integrações com outros serviços, sem a necessidade de alterar diretamente os manifestos que são incluídos no *chart*.

Um exemplo comum de uso do Helm ocorre na instalação de aplicações de terceiros, como os bancos de dados MySQL e PostgreSQL, ou ferramentas de monitoramento, como Prometheus e Grafana, pois os charts estão disponíveis em repositórios. Além disso, soluções como o Vault, da HashiCorp, dispõem de charts oficiais. Devida a essa facilidade, o uso do Helm, simplifica e acelera a implantação de ferramentas complexas em *cluster* Kubernetes.

2.3.1 Repositórios de Helm Charts

Um repositório de charts é uma biblioteca destinada ao armazenamento de pacotes do Helm. Atualmente, o Artifact Hub funciona como um catálogo público que indexa diversos repositórios de *charts*, os quais podem estar hospedados em plataformas como ChartMuseum, GitLab ou GitHub.

O Artifact Hub é uma aplicação web que permite divulgar, encontrar e instalar pacotes, facilitando o compartilhamento e a integração de *charts* dentro do ecossistema Kubernetes (ARTIFACT, 2025).

O Artifact Hub é uma plataforma mantida pela CNCF (Cloud Native Computing Foundation) que funciona como um catálogo público e centralizado de Helm charts e outros artefatos relacionados ao Kubernetes. Em vez de armazenar diretamente os pacotes, o Artifact Hub indexa e organiza repositórios externos, permitindo que desenvolvedores e equipes encontrem, avaliem e utilizem facilmente aplicações de terceiros, como bancos de dados, sistemas de monitoramento, ferramentas DevOps[5], entre outros. Através do Artifact Hub, é possível pesquisar por *charts* verificados, consultar documentações, acompanhar atualizações e avaliar a confiabilidade dos pacotes.

Para casos em que se deseja manter *charts* privados ou personalizados, como aplicações internas de uma organização, uma solução amplamente adotada é o Chartmuseum. Trata-se de um repositório Helm *self-hosted*, que permite armazenar e servir *charts* por meio de uma API,[6] (*Application Programming Interface*) compatível com o Helm e o Argo CD (Continuous Delivery). O Chartmuseum oferece suporte a autenticação e controle de acesso, sendo ideal para ambientes corporativos que exigem segurança e controle sobre seus pacotes (CHARTMUSEUM, 2025).

2.4 Registro de Imagens de *Containers*

Docker Hub é um repositório público de *imagens de container*[7], fornecido pela Docker, Inc., e funciona como uma plataforma centralizada para armazenar, versionar e compartilhar imagens. Ele oferece acesso a milhares de imagens públicas, incluindo as ofici-

[5]Conjunto de práticas, cultura e ferramentas que combina desenvolvimento (Dev) e operações de TI (Tecnologia da Informação) (Ops) para acelerar a entrega de software de alta qualidade e confiabilidade

[6]É um conjunto de regras e protocolos que permite que diferentes softwares se comuniquem e troquem informações entre si

[7]Essas imagens servem como modelos para criar *containers* executáveis de forma padronizada e reproduzível.

ais e verificadas, que garantem maior segurança e confiabilidade, além de permitir que os desenvolvedores publiquem suas próprias imagens em repositórios públicos ou privados (DOCKER, 2025).

O Harbor é uma plataforma *open source* para registro, armazenamento e distribuição de imagens de *containers*, desenvolvida originalmente pela VMware[8] e atualmente é mantida pela CNCF. Ele funciona como um registro privado de imagens, com foco em segurança, desempenho e gerenciamento corporativo (CNCF, 2025b).

O Harbor oferece recursos avançados que vão além de um registro tradicional de imagens, proporcionando maior controle, segurança e organização. Ele implementa controle de acesso baseado em funções RBAC (Role-Based Access Control), permitindo definir permissões granulares para usuários e equipes. Oferece suporte a múltiplos projetos, o que facilita a segregação e o gerenciamento de imagens de acordo com diferentes aplicações ou grupos de trabalho. Inclui escaneamento automático de vulnerabilidades, integrando ferramentas como o Trivy[9] para identificar falhas de segurança em imagens de *containers*. Além disso, possibilita fazer cache de imagens a partir de outros repositórios.

2.5 Argo CD

O Argo CD (Continuous Delivery) é uma ferramenta *open source* voltada para a entrega contínua de aplicações em *cluster* Kubernetes, fundamentada nos princípios de GitOps[11]. Desenvolvido como parte do projeto Argo da CNCF, o Argo CD permite que o estado desejado dos sistemas em Kubernetes seja descrito de forma declarativa em repositórios Git, que passam a ser a fonte de verdade (*source of truth*) para o *cluster* (CNCF,

[8]VMware é uma empresa de virtualização e computação em nuvem

[9]Trivy é um scanner de vulnerabilidades que identifica falhas em imagens de *container*, repositórios Git[10], pacotes do sistema, bibliotecas da aplicação, configurações de Kubernetes e arquivos IaC (Infrastructure as Code).

[11]GitOps é um modelo operacional que utiliza repositórios Git como fonte única de verdade para declarar e versionar o estado desejado de infraestruturas e aplicações, permitindo que todo o ciclo de mudanças seja rastreável, auditável e reproduzível.

2025a).

Monitorando continuamente os projetos Git, o Argo CD compara o estado atual do serviço com o estado desejado descrito nos documentos versionados. Ao identificar divergências, a ferramenta pode notificar os usuários, sincronizar o *cluster* com o estado definido no repositório e registrar o histórico completo das alterações. Essa abordagem proporciona auditabilidade e versionamento, uma vez que todas as mudanças ficam documentadas no Git, além de facilitar o *rollback*[12], permitindo retornar a versões anteriores diretamente pelo repositório ou pela interface de usuário da ferramenta.

2.5.1 *Backup* e Recuperação de Desastres no Argo CD

O Argo CD disponibiliza mecanismos nativos que facilitam significativamente os processos de *backup* e *disaster recovery*. Dessa forma, mesmo em situações de falha total do *cluster* Kubernetes, a restauração dos serviços pode ser realizada.

Possui mecanismos para exportar e importar o estado do Argo CD em documentos, contendo sua configuração e a dos serviços que, por meio dele, são instalados no *cluster*. Esse artefato contém todas as informações necessárias para restaurar o ambiente e suas aplicações em um novo *cluster* Kubernetes. Com ele, o Argo CD consegue reconstituir as configurações e ressincronizar os serviços que se associam a ele, o que evidencia sua versatilidade em cenários adversos.

2.6 OpenTelemetry

O OpenTelemetry é um projeto de código aberto que fornece um conjunto de ferramentas, APIs e SDKs[13] para coletar, gerar, processar e exportar dados de telemetria de aplicações. Ele facilita a observabilidade dos sistemas, permitindo que desenvolvedores

[12]Processo de reversão que restaura um sistema ao seu estado anterior

[13]SDKs, ou Kits de Desenvolvimento de Software, são pacotes de ferramentas que ajudam programadores a criar aplicativos para uma plataforma específica

monitorem o desempenho e o comportamento das aplicações de forma padronizada, independentemente da linguagem ou da infraestrutura usada. É um projeto mantido pela CNCF, resultado da fusão entre o OpenTracing e o OpenCensus (OPENTELEMETRY, 2025a).

O OpenTelemetry Collector é um componente central da arquitetura OpenTelemetry, que atua como um agente ou *gateway* para receber, processar e exportar dados de observabilidade, como *traces*, métricas e *logs* (OPENTELEMETRY, 2025b).

O Collector desempenha um papel essencial na arquitetura de observabilidade, atuando como um intermediário flexível entre as fontes de dados e as ferramentas de análise. Ele é responsável pela recepção de dados, coletando informações de diversas origens e protocolos, como gRPC (Google Remote Procedure Call), HTTP (Hypertext Transfer Protocol) e OTLP (OpenTelemetry Protocol). Em seguida, realiza o processamento, aplicando transformações, filtragens, agregações, amostragens e enriquecimentos aos dados antes de seu encaminhamento. Por fim, executa a exportação, enviando as informações tratadas para destinos como Prometheus, Jaeger, Zipkin, Grafana Tempo e outras plataformas compatíveis, garantindo assim um fluxo contínuo e padronizado de métricas, *logs* e *traces*.

O OpenTelemetry Collector é o serviço responsável por orquestrar o recebimento, processamento e exportação dos dados de observabilidade. Ao utilizar *connectors* e *processors*, ele consegue transformar dados em informações estruturadas, viabilizando a construção de painéis como o de OpenTelemetry APM (Application Performance Management), que será abordado na seção 2.6.1.

O OpenTelemetry Operator é um recurso desenvolvido para o Kubernetes que automatiza a instalação, configuração e gerenciamento dos componentes do OpenTelemetry dentro de um *cluster*. Ele é capaz de injetar automaticamente SDKs de instrumentação nos *containers* das aplicações. Para isso, utiliza *Mutating Admission Webhooks*, que modificam o *PodSpec* no momento da criação do *pod*, adicionando Sidecars ou Init-containers

contendo os agentes de instrumentação, além de definir as variáveis de ambiente e configurações necessárias para o envio da telemetria. Essa abordagem elimina a necessidade de alterar o código-fonte da aplicação, tornando o processo de instrumentação mais simples, padronizado e menos suscetível a erros manuais (OPENTELEMETRY, 2025e).

O OpenTelemetry Operator é responsável por injetar e configurar bibliotecas de instrumentação automática para serviços .NET, Java, Node.js, Python e Go (OPENTELEMETRY, 2025d).

Grande parte das linguagens de programação oferece bibliotecas para a inclusão do OpenTelemetry Exporter. Mas em sistemas legados ou soluções de terceiros onde não se tem acesso ao código-fonte, a auto-instrumentação surge como uma alternativa prática que permite instrumentar serviços sem configuração prévia.

2.6.1 Application Performance Management

O componente Span Metrics Connector, utilizado pelo Opentelemetry Collector, disponível no projeto OpenTelemetry Collector Contrib é responsável por converter traces (*spans*) em métricas do tipo RED (Requests, Errors, Duration). Esse *conector* atua simultaneamente como exportador no pipeline de *traces* e como *receptor* no pipeline de métricas, permitindo que os dados de execução das aplicações sejam transformados em métricas agregadas (OPENTELEMETRY, 2025c). Com os dados gerados, é possível construir o *dashboard* de OpenTelemetry APM (Application Performance Management) no Grafana, o que possibilita visualizar de forma completa o comportamento de uma aplicação ao longo do tempo, reunindo métricas de tráfego, desempenho e estabilidade. Ele mostra o volume de requisições recebidas, a taxa de erros e a distribuição dos diferentes códigos de resposta, permitindo avaliar rapidamente a saúde geral do serviço. Também exibe a forma como o tráfego se distribui entre as rotas da aplicação, destacando quais *endpoints* são mais acessados e quais concentram maior carga de processamento. Além disso, o painel apresenta métricas de latência, tanto médias quanto percentis, que ajudam

a identificar eventuais lentidões, bem como gráficos de taxa de requisições que mostram a evolução temporal do uso. Por fim, inclui uma tabela detalhada por rota, consolidando informações de requisições, erros e tempos de resposta para facilitar a análise de cada *endpoint* individual.

2.7 Coroot

O Coroot é uma plataforma de observabilidade que combina métricas, *logs* e rastreamentos para oferecer uma visão completa da saúde e do desempenho de sistemas hospedados no *cluster*. Ele coleta métricas de desempenho por meio do Prometheus, monitorando o tempo de resposta, a utilização da CPU, memória, tráfego de rede e as dependências entre os serviços. Com base nesses dados, o Coroot detecta automaticamente gargalos e regressões de performance, reduzindo o tempo de diagnóstico (COROOT, 2025).

Coroot traz algumas abordagens relacionadas ao monitoramento dos *nodes*, mas dentre suas características, se destaca o monitoramento de serviços hospedados, trazendo funcionalidades importantes como o rastreamento de serviços sem instrumentação, por meio do eBPF (Extended Berkeley Packet Filter)[14]. Essa abordagem permite rastrear chamadas entre serviços sem modificar o código das aplicações, o que facilita a adoção do rastreamento distribuído em sistemas que não possuem OpenTelemetry Exporter.

O Coroot também permite correlacionar *logs*, métricas e rastreamentos, oferecendo uma visão unificada dos eventos e acelerando a análise da causa raiz. Sua interface intuitiva apresenta *dashboards* automáticos que exibem os dados observáveis dos serviços.

Além disso, o Coroot realiza a detecção proativa de problemas, utilizando regras inteligentes e análises automáticas para alertar sobre situações críticas, como latência elevada, falhas em dependências ou uso anormal de recursos pelos *pods*, possibilitando ações cor-

[14]É uma tecnologia do kernel Linux que permite executar programas de forma segura, isolada e altamente eficiente dentro do próprio kernel, sem precisar modificar o código-fonte do kernel ou carregar módulos tradicionais.

retivas rápidas e eficazes por meio dos alertas de monitoramento.

A versão *Enterprise* do Coroot oferece recursos exclusivos como análise de causa raiz com AI (Artificial Intelligence), SSO (Single Sign-On) corporativo, RBAC granular com papéis personalizados e implantação totalmente *self-hosted*, garantindo controle total dos dados.

2.8 Netdata

O Netdata conforme (NETDATA, 2025b), possui a capacidade de monitorar detalhadamente os processos em execução em uma máquina, permitindo identificar o consumo da CPU, memória, uso de disco e rede de forma granular, por processo em execução. Além disso, realiza a detecção automática de diversos serviços instalados no sistema, incluindo servidores *web* como Apache e Nginx, bancos de dados como MySQL e PostgreSQL, além de sistemas de cache como Redis, ferramentas de mensageria como RabbitMQ e outros componentes essenciais à infraestrutura. Essa capacidade de autodetecção elimina a necessidade de configurações.

Pode trazer funções relacionadas aos sistemas hospedados em Kubernetes, entretanto o diferencial notado é o monitoramento dos hosts, pois, além do monitoramento individual de processos, realiza um ranqueamento dinâmico dos processos que mais consomem recursos da VM (Virtual Machine). Esse *ranking* apresenta, em tempo real, os processos que utilizam a maior quantidade da CPU, memória RAM (Random Access Memory) e largura de banda de rede, permitindo uma análise rápida das aplicações ou serviços que mais impactam o desempenho do sistema.

A versão paga do Netdata oferece métricas, *logs*, painéis personalizados e retenção de dados ilimitada, além do Netdata AI para investigação automática, solução de problemas e geração de relatórios. Inclui recursos corporativos como RBAC com permissões refinadas, SSO, SCIM (System for Cross-domain Identity Management), monitoramento

de Windows, gerenciamento centralizado de configuração, integrações avançadas de notificações (MS Teams, Slack, OpsGenie, PagerDuty, etc.) e registros de auditoria para segurança e conformidade. A edição *Enterprise On-Premise*, indicada para organizações que precisam de total isolamento de dados, hospeda todos os componentes internamente e é ideal para ambientes críticos ou isolados de rede.

2.9 Netdata vs Coroot

Desenvolvedores do Netdata (NETDATA, 2025a) relatam algumas comparações entre Coroot e Netdata. Algumas são pertinentes a serem abordadas. O Coroot baseia-se fortemente em eBPF para oferecer observabilidade sem instrumentação, com foco especial em ambientes Kubernetes. Ele fornece mapas de serviços, análise de *logs* e alertas orientados a SLOs (Service Level Objectives), sendo uma solução eficiente para equipes centradas em aplicações distribuídas dentro de *clusters*.

O Netdata, por outro lado, destaca-se por sua abordagem distribuída e orientada ao *edge*[15], utilizando um agente leve instalado em cada *node*. Esse modelo permite a coleta e análise local com granularidade de um segundo, garantindo desempenho elevado e segurança ao manter dados sensíveis dentro da infraestrutura do usuário. Além disso, o Netdata executa *machine learning* não supervisionado em cada métrica e em cada *node*. Possui centenas de integrações automáticas, indo além de Kubernetes e abrangendo *bare metal*, VM, *containers* e IoT (*Internet of Things*).

Assim, enquanto o Coroot se sobressai em visibilidade de *workloads* Kubernetes via eBPF, o Netdata apresenta abrangência para o monitoramento de máquinas, ideal para os *nodes* que compõem o *cluster* Kubernetes. Mais dados comparativos podem ser vistos na Tabela A.5 anexada em Apêndice.

[15] Arquitetura orientada a edge (ou edge-oriented architecture) significa que o Netdata foi projetado para executar a coleta, processamento e análise de métricas no próprio local onde os dados são gerados.

2.10 Prometheus

O Prometheus é uma solução de monitoramento de código aberto amplamente utilizada para acompanhar aplicações, sistemas e serviços. Ele permite instrumentar, coletar, armazenar e consultar métricas de desempenho, oferecendo recursos como geração de alertas, visualização em *dashboards* e análises detalhadas (PROMETHEUS, 2025).

O Prometheus Server é o componente central da arquitetura Prometheus, sendo responsável por coletar, armazenar e consultar métricas de sistemas e aplicações, atuando como o núcleo do sistema de monitoramento. Complementarmente, o Prometheus Exporter funciona como um componente auxiliar que se relaciona com o sistema a ser monitorado e expõe métricas no formato compreendido pelo Prometheus Server, geralmente por meio de um *endpoint* HTTP.

Integração do Prometheus com sistemas de gerenciamentos

O Prometheus oferece uma arquitetura flexível e extensível que permite a integração com aplicações personalizadas. Essa característica possibilita o desenvolvimento de utilidades avançadas, como sistemas de previsão de métricas, detecção automática de anomalias e alertas inteligentes baseados em aprendizado de máquina.

Segundo (MART et al., 2020), o autor propõe exatamente essa abordagem ao desenvolver uma aplicação de predição que consome dados do Prometheus, gera previsões e as reenvia como novas métricas para análise:

> "We need a prediction application that should take data from Prometheus, based on that data our app should create a model which will produce a forecast and send it back to Prometheus as another metric."

Na conclusão do estudo, os autores reforçam que a integração entre Prometheus e ferramentas preditivas amplia significativamente o potencial de observabilidade e automação

dos sistemas em nuvem, permitindo o uso de métodos totalmente automáticos de previsão de métricas:

> "We implemented a solution for forecasting Prometheus time series data, that can take advantage of Kubernetes scalability features." (MART et al., 2020)

Dessa forma, o Prometheus deixa de atuar apenas como ferramenta de monitoramento e passa a compor um ecossistema inteligente de observabilidade, capaz de identificar anomalias e ajustar-se dinamicamente a comportamentos inesperados nos *clusters* Kubernetes.

2.11 Grafana

O Grafana é uma plataforma *open-source* de visualização e análise de métricas. Ele permite criar *dashboards* interativos a partir de dados de diferentes fontes, ajudando equipes a monitorar sistemas, identificar anomalias e tomar decisões baseadas em dados (GRAFANA, 2025a).

O Grafana apresenta diversas características que o tornam uma ferramenta robusta para visualização e monitoramento de dados. Ele oferece suporte a múltiplos *datasources*, incluindo Prometheus, InfluxDB, Elasticsearch, Loki, entre outros, permitindo a integração com diferentes fontes de métricas e *logs*. Além disso, possibilita a criação de painéis personalizados com gráficos, tabelas, mapas de calor e alertas, proporcionando uma análise visual completa das informações. O Grafana também permite configurar alertas visuais e notificações em tempo real, garantindo que as equipes sejam imediatamente informadas sobre qualquer anomalia ou mudança significativa nos sistemas monitorados.

2.12 Grafana Tempo

O Grafana Tempo é um sistema de código aberto projetado para armazenar e consultar rastreamentos de aplicações distribuídas. Em ambientes modernos baseados em microsserviços, onde uma única requisição pode atravessar diversos serviços e camadas, torna-se fundamental identificar gargalos, latências e dependências entre os componentes do sistema. O Grafana Tempo contribui com essa necessidade ao armazenar e indexar rastreamentos completos de transações (GRAFANA, 2025b).

É compatível com os principais protocolos de rastreamento, como OpenTelemetry, Jaeger e Zipkin, o que facilita sua adoção em arquiteturas já existentes e se integra perfeitamente ao Grafana.

2.13 HashiCorp Vault

O HashiCorp Vault é uma ferramenta de código aberto desenvolvida para realizar o gerenciamento seguro de segredos, como senhas, *tokens* de API, certificados digitais e credenciais de banco de dados. Ele centraliza o armazenamento desses dados de forma criptografada, oferecendo mecanismos avançados de autenticação, controle de acesso baseado em políticas ACLs (Access Control List) e auditoria de operações. O principal objetivo do Vault é proteger informações sensíveis e garantir que apenas aplicações e usuários autorizados possam acessá-las (HASHICORP, 2025).

O Vault suporta múltiplos métodos de autenticação, incluindo LDAP (Lightweight Directory Access Protocol), *tokens*, *GitHub* e, particularmente, o método de autenticação via Kubernetes, o qual é essencial para ambientes baseados em microsserviços e orquestração de *containers*.

Para ambientes que utilizam o Kubernetes como plataforma de execução de aplicações, a HashiCorp fornece o Vault Agent Injector, um componente que facilita a injeção automática de segredos diretamente em *pods*. Esse processo é realizado por meio de um

Mutating Admission Webhook, que intercepta solicitações de criação de *pods* e insere automaticamente um sidecar *container* (Vault Agent) no *pod*.

O Vault Agent realiza a autenticação no Vault e, em seguida, recupera os segredos definidos nas anotações do manifesto. Esses segredos são disponibilizados como arquivos em um volume compartilhado com o *container* principal da aplicação, garantindo que os dados sensíveis não fiquem expostos diretamente em variáveis de ambiente ou objetos *Secrets* do Kubernetes.

Essa integração com o *Kubernetes* permite o acesso a *secrets* de forma segura, dinâmica e alinhada às melhores práticas de segurança da informação, seguindo o princípio do menor privilégio. Esse processo garante que os segredos sejam acessados exclusivamente em tempo de execução, reduzindo significativamente sua exposição. Dessa forma, apenas os *pods* autorizados e, quando necessário, profissionais devidamente habilitados têm acesso às informações sensíveis.

Capítulo 3

Desenvolvimento

3.1 Implementação do sistema

Para a implementação do sistema, inicialmente criou-se um *cluster* Kubernetes e, em seguida, o Argo CD foi instalado por meio do Helm, utilizando o *chart* listado no Apêndice pela Tabela A.4. Os demais serviços foram implantados por meio do CRD *Application*, reconhecível pelo Argo CD.

O recurso *Application*, exemplificado no Apêndice pela Figura A.2, permite definir, de forma declarativa, os sistemas a serem instalados, com especificações do repositório do tipo Git ou *chart* de onde o Argo CD obtém as configurações que descrevem o estado desejado do sistema. Possibilita incluir variáveis de customização, versão do *chart* e configurações de sincronização do serviço.

O Argo CD garante que o estado dos sistemas ilustrado na Figura 3.1 permaneça alinhado com as configurações declaradas no repositório Git. Na interface IDE, apresentada na Figura 3.2, é possível visualizar todos os serviços, seu estado operacional e o nível de sincronização com o repositório.

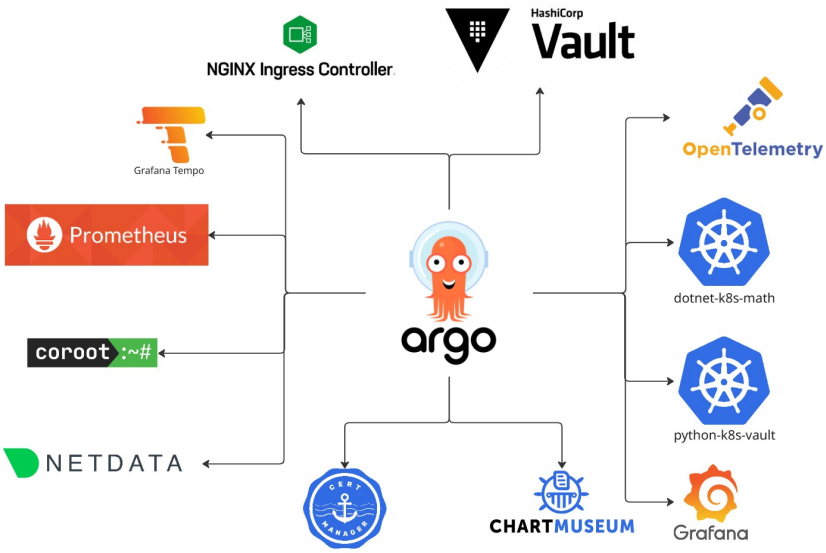

Figura 3.1: Sistemas instalados a partir do Argo CD

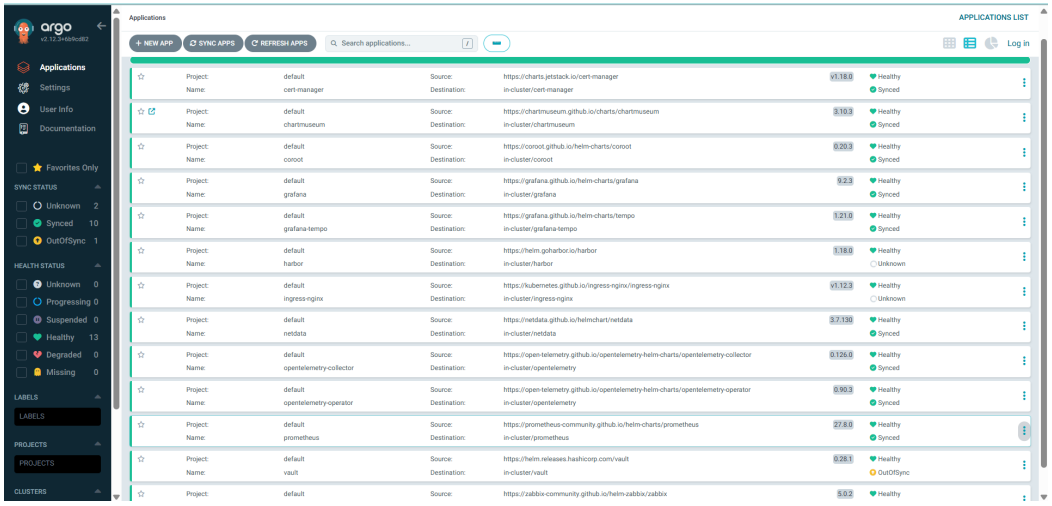

Figura 3.2: IDE do Argo CD

3.2 Arquitetura do sistema

Nesta seção, descreve-se como os fluxos de dados ocorrem, desde a coleta de métricas até o processamento, incluindo *logs*, *traces* e *charts*, evidenciando como as ferramentas

se integram. As principais contribuições de cada ferramenta para a arquitetura podem ser vistas na Tabela A.1, que está no Apêndice.

O OpenTelemetry Operator é responsável por inserir configurações nas aplicações para habilitar a coleta de dados observáveis, como métricas e *traces*. Ele injeta variáveis de ambiente e configura o OpenTelemetry Exporter (parte do Instrumentation SDK ou do agente de auto-instrumentação), que coleta e envia esses dados para o OpenTelemetry Collector.

O OpenTelemetry Collector é responsável por receber e processar os dados de telemetria gerados pelas aplicações, permitindo a utilização de componentes como o Span Metrics Connector, que converte traces em métricas. As métricas resultantes são armazenadas no *Prometheus* Server e são utilizadas na construção dos *dashboards* no Grafana de OpenTelemetry APM como pode ser visto na Figura 4.2. Além disso, o OpenTelemetry Collector encaminha os dados para o Coroot e o Grafana Tempo, onde são armazenados e disponibilizados para análises históricas e diagnósticos detalhados.

O Netdata utiliza agentes instalados em cada *node* do *cluster* para coletar métricas de desempenho e gerar *dashboards* de forma autônoma. A ferramenta é capaz de identificar automaticamente diversos sistemas em execução na máquina virtual. Por exemplo, ao detectar um Nginx, o Netdata coleta suas métricas específicas e cria um *dashboard* dedicado. Esse comportamento se estende a uma ampla variedade de serviços e aplicações suportados, permitindo uma observabilidade imediata e detalhada sem necessidade de configuração manual. No contexto do Kubernetes, o Netdata monitora tanto o *cluster* quanto os recursos individuais, como *pods*, *containers*, *namespaces* e *nodes*, oferecendo uma visão unificada do desempenho e do consumo de recursos. Além disso, sua principal contribuição é a sua variedade de alertas pré-configurados, que monitoram continuamente o estado do *node* e notificam em caso de anomalias. Esses alertas podem ser personalizados conforme as necessidades do ambiente, garantindo um monitoramento mais preciso, proativo e integrado à infraestrutura do Kubernetes.

O Coroot atua no monitoramento dos sistemas hospedados de forma individualizada e gera indicadores como SLO, *logs* agrupando por similaridade, registro de incidentes como OOM KILL (Out Of Memory Kill), *memory lake* e outros que também estão registrados na Tabela A.2. Oferece dois tipos de *traces*: o baseado em traces recebidos pelo OpenTelemetry Collector e o baseado em *traces* via eBPF. O primeiro tipo está associado diretamente às aplicações instrumentadas, permitindo correlacionar as métricas e os *traces* recebidos. Essa integração proporciona uma observabilidade em ambientes que utilizam o OpenTelemetry Exporter. Já o *trace* baseado em eBPF coleta dados diretamente do *kernel* do Linux, sem a necessidade de modificar o código das aplicações. Essa abordagem não intrusiva é ideal para a hospedagem de sistemas em que o OpenTelemetry Exporter ainda não está implementado, pois garante uma visibilidade essencial sobre o comportamento e as interações das aplicações, assegurando o monitoramento eficiente mesmo em sistemas sem instrumentação prévia.

O Docker Hub é utilizado como repositório de imagens de *container*. O *container* de aplicação também foi armazenado nele para que pudesse ser utilizado na criação do *Pod*.

O Prometheus instalado no *cluster* Kubernetes atua com componentes de coleta e armazenamento de métricas. Possui agentes que coletam dados no *host*, abrangendo informações do sistema operacional, como uso da CPU, memória, disco e rede, além de dados detalhados sobre *pods*, *containers*, *namespaces*, *nodes* e outros recursos nativos do Kubernetes. Essas métricas, integradas ao Grafana, permitem a criação de *dashboards* ricos em informações, oferecendo visibilidade completa sobre os recursos operacionais do *cluster*. O *dashboard* utilizado para apresentação dos dados consta na Tabela A.3.

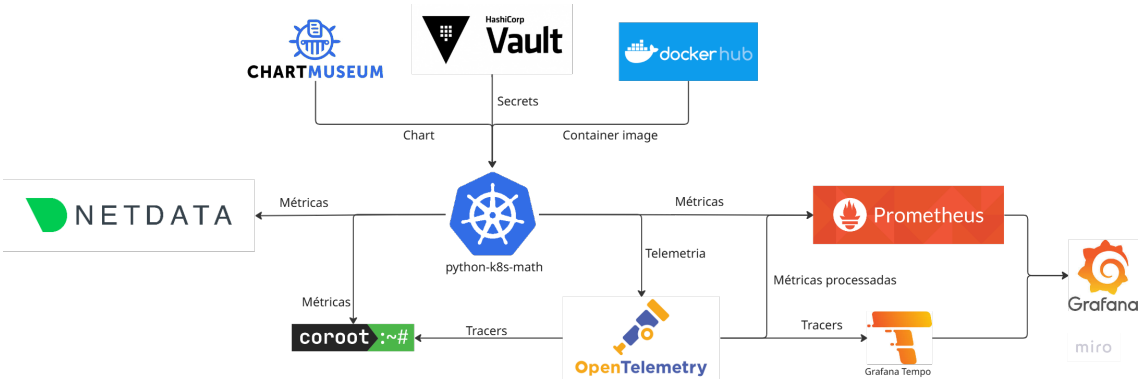

Figura 3.3: Fluxo de dados entre os sistemas

O Vault é responsável por armazenar as *secrets* que são injetadas nos *containers* por meio de um sidecar, permitindo que sistemas localizados em diferentes *Namespaces* e que possuam *Service Accounts* autorizadas tenham acesso seguro a essas informações.

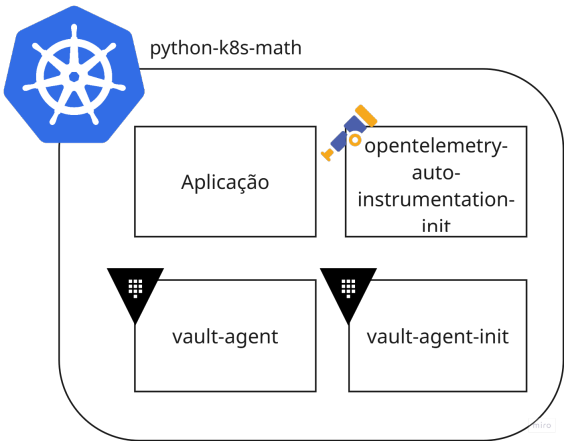

Figura 3.4: *Containers* que compõem um *pod*

O *pod* de um serviço, pode possuir quatro *containers*: o da aplicação principal e, quando necessário, os *containers* auxiliares vault-agent, vault-agent-init e opentelemetry-auto-instrumentation-init.

O vault-agent-init é um *init container* responsável por preparar o ambiente seguro antes que a aplicação inicie. Ele autentica o *Pod* no Vault e busca os segredos necessários, como senhas, *tokens* e certificados, armazenando-os em um volume compartilhado. Esse

processo garante que, quando a aplicação for iniciada, já tenha acesso às credenciais para coletar as *secrets* de forma segura, sem precisar expor informações sensíveis.

O vault-agent, por sua vez, é um sidecar *container* que permanece em execução durante todo o ciclo de vida do *Pod*. Seu papel é manter os segredos atualizados, renovando *tokens* e certificados automaticamente conforme se aproximam da expiração. Isso evita que a aplicação precise lidar diretamente com a renovação de credenciais, tornando a segurança mais dinâmica e automatizada.

Como descrito em (VAULT, 2019), o mecanismo de Vault Init garante que as *secrets* estejam disponíveis na inicialização do sistema, enquanto o *Vault Agent* é responsável por atualizá-las dinamicamente durante o tempo de execução sempre que houver alterações nos dados sensíveis armazenados.

O opentelemetry-auto-instrumentation-init, também um *init container*, prepara a camada de observabilidade do sistema. Ele instala e configura as bibliotecas de auto-instrumentação para que o OpenTelemetry Exporter envie dados para o OpenTelemetry Collector. Assim, quando o *container* principal iniciar, sua instrumentação estará apta.

O *container* da aplicação principal é responsável por executar a lógica do sistema. Ele consome as *secrets* injetadas e utiliza as configurações de telemetria para seu funcionamento.

Capítulo 4

Experimentos e Resultados

Os experimentos foram conduzidos por meio da implantação de três tipos de serviços no cluster Kubernetes. O primeiro consiste em uma aplicação destinada a cálculos matemáticos; o segundo envolve a execução de testes de estresse e seguido de uma simulação de saturação da capacidade de armazenamento do node; por fim, o terceiro corresponde à implantação de um serviço que simula latência e erros.

Ao final dos testes, incluiu-se uma seção com informações adicionais que podem ser obtidas pelas ferramentas a respeito da infraestrutura e dos sistemas hospedados.

4.1 Caso 1 - Calculadora com histórico em banco de dados

Para analisar a aquisição de *secrets* no Vault, dados em *dashboards* no Grafana e no Coroot, foi desenvolvida uma calculadora utilizando *Python*, que possibilita a autoinstrumentação. O sistema se comunica com um banco de dados MongoDB para armazenar o histórico dos cálculos, cuja interface de usuário é mostrada na Figura 4.1. Para a aquisição de métricas, foram definidos dois contextos: no primeiro, a senha do banco de dados foi cadastrada corretamente no Vault e a calculadora pôde operar normalmente, nesse caso, foram inseridas expressões válidas e inválidas. No outro contexto, foi cadastrada uma senha incorreta no Vault, com o intuito de gerar *logs* de erro e indisponibilidade.

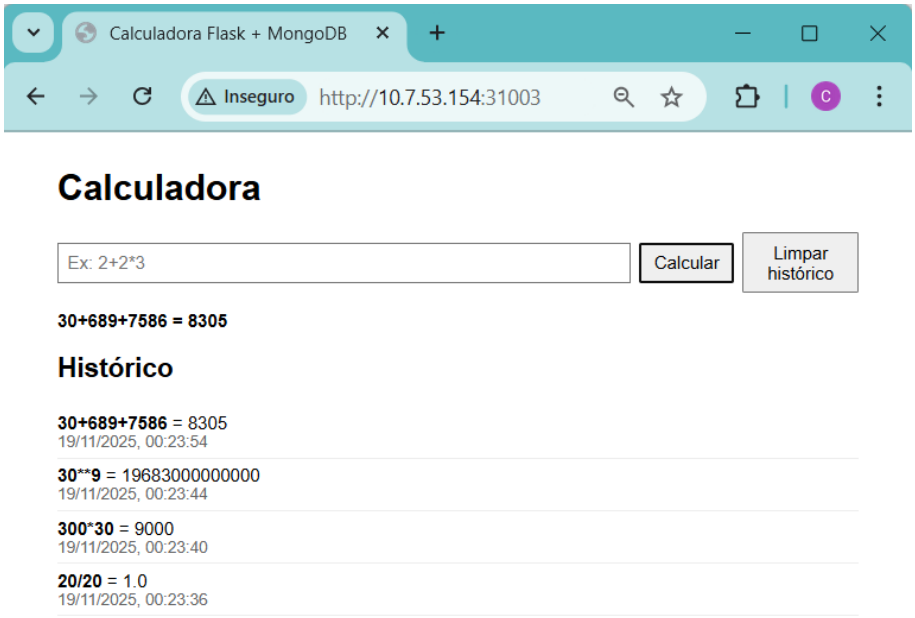

Figura 4.1: Visão geral do sistema Calculadora

Após produzir os dados com os *inputs*, foram gerados indicadores no dashboard do OpenTelemetry APM como pode ser visto na Figura 4.2. A distribuição apresentada de requisições evidencia que o endpoint POST /calculate é o mais acionado, representando 55% do tráfego, seguido pelo GET /history (43%) e, em menor escala, pelo GET / (3%). A distribuição de carga (Loading Distribution) mostra que o processamento também está concentrado quase exclusivamente no endpoint /calculate, responsável por 96% do esforço computacional. Isso indica que essa rota é significativamente mais custosa em termos de recursos, possivelmente devido a cálculos, acesso ao banco ou lógica mais complexa.

Nos gráficos de Request Rate, é possível observar momentos de pico, principalmente no endpoint /calculate, que chega a registrar entre 3 e 4 requisições por minuto. Paralelamente, o gráfico de PR95 Latency, que mostra o limite de tempo que 95% das requisições mais rápidas levam para serem concluídas, revela que o endpoint /calculate apresenta as maiores latências, chegando a ultrapassar 250 ms em determinados momentos, enquanto os endpoints / e /history mantêm latências muito baixas, próximas de zero. Isso reforça

que o gargalo de desempenho se concentra em /calculate.

O painel de Average Latency reforça essa diferença: requisições bem-sucedidas para POST /calculate apresentam uma média de aproximadamente 10 ms, enquanto o erro 500 para o mesmo endpoint atinge 381 ms, indicando que as falhas estão associadas a operações demoradas. Já os endpoints GET / e GET /history mantêm tempos médios estáveis e baixos, variando entre 4 e 7 ms.

Por fim, a seção Details resume as métricas individuais de cada rota, confirmando que /calculate possui a maior latência PR95 (5,77 ms) e a taxa de erro mais significativa (27,3%), enquanto as demais rotas apresentam desempenho estável e sem registros de falhas relevantes.

Gráficos como este evidenciam os pontos que geram maior latência nos serviços, afetando diretamente o usuário final. Com as informações obtidas, é possível direcionar recursos humanos ou computacionais para aprimorar o ponto específico, seja por meio de otimização de código, correção de *bugs* ou até mesmo pelo redirecionamento de parte da carga processada pelo *endpoint* para novas rotas ou instâncias. Outra vantagem é a capacidade de comparar o progresso ao longo do tempo, permitindo avaliar numericamente a evolução e o aprimoramento dos serviços.

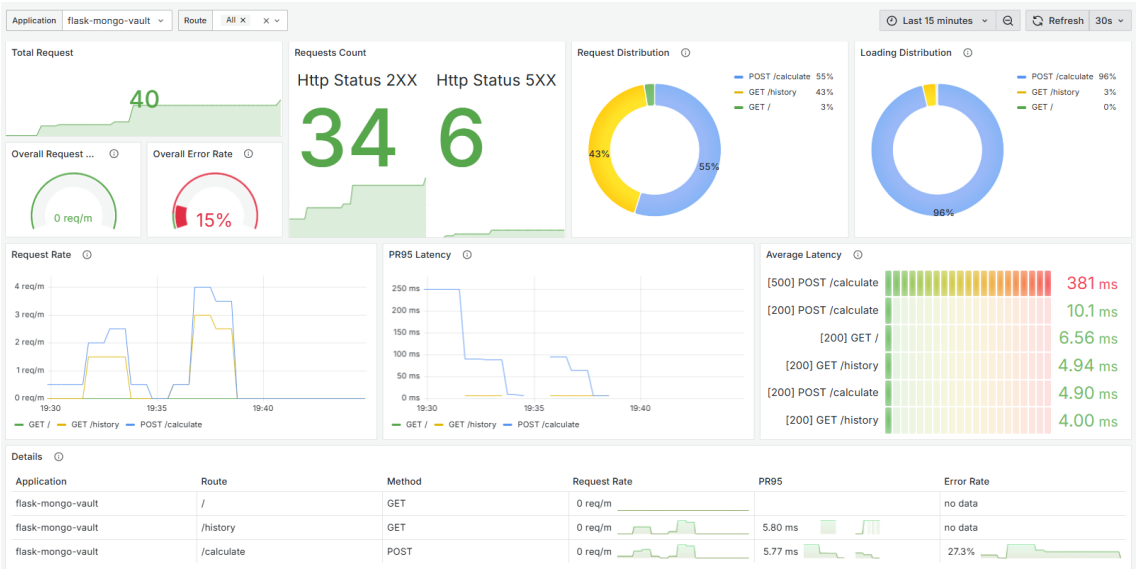

Figura 4.2: Gráfico Open Telemetry APM

Com a análise de traces pelo Grafana vinculado ao Grafana Tempo elucidado na Figura 4.3, pode-se ver os traces associados aos sistemas que possuem telemetria.

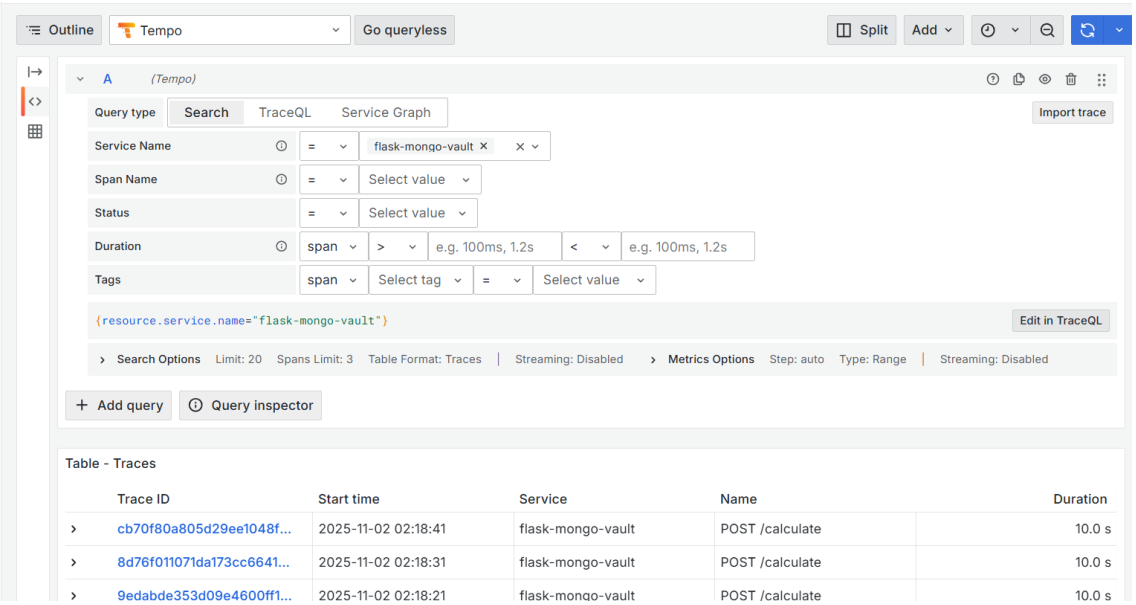

Figura 4.3: Grafana Tempo no Grafana

Pode-se observar, na Figura 4.4, a análise de um trace da aplicação calculadora realizada no Grafana, utilizando dados armazenados no Grafana Tempo. Traz informações

referentes a uma requisição POST no *endpoint* /calculate, que levou um tempo de 7,01 ms do início ao fim. Cada barra representa uma etapa da execução. O primeiro *span*, HTTP *receive*, teve duração de 41,66 µs, indicando o tempo necessário para o serviço receber os dados da requisição. O *span* mais significativo é calculator.insert, com duração de 2,89 ms, representando o tempo gasto na operação de inserção no banco de dados e sendo responsável pela maior fatia do tempo total da requisição. Em seguida, aparecem três *spans* curtos relacionados ao HTTP *send*, que correspondem às pequenas etapas internas de envio da resposta ao cliente. Os espaços em branco entre as barras representam períodos não rastreados pelo código, ou seja, trechos da execução que não possuem *spans* instrumentados. Com isso, podemos ter uma visão mais interna e identificar os gargalos que contribuem para maior latência de um *endpoint*, possibilitando identificar dependências que mais impactam no tempo de resposta.

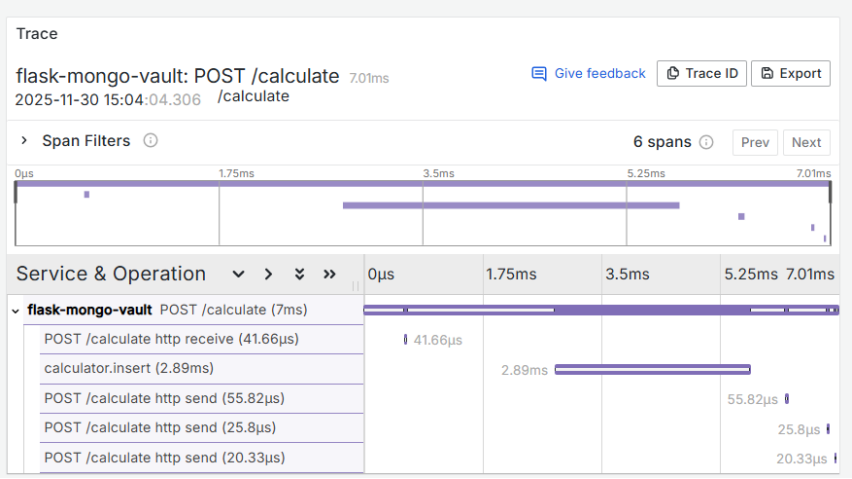

Figura 4.4: Inspeção de trace

No Coroot é possível ver o fluxo de dados através de um gráfico, demonstrado na Figura 4.5.

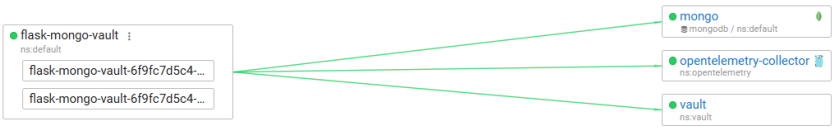

Figura 4.5: Mapa do fluxo de rede

Para avaliar os *logs*, simulamos alguns erros, como ao inserir uma senha incorreta do banco no Vault e ao inserir expressões matemáticas que são consideradas inválidas. O Coroot armazena os *logs* dos *pods* e cria conjuntos de *logs* por similaridade, mostrando a porcentagem de ocorrência de cada grupo, como pode ser visto na Figura 4.6.

Figura 4.6: Listagem de *logs*

4.2 Caso 2 - Testes de sobrecarga

Sobrecarga de CPU e memória

Para avaliar o comportamento do *cluster* diante de cenários de estresse de recursos, foi utilizado um *container* configurado para consumir integralmente a CPU e a memória que lhe foram atribuídas. Foi empregado o stress-ng, projeto amplamente utilizado para gerar cargas sintéticas sobre diferentes subsistemas do sistema operacional. O stress-ng

permite configurar múltiplos tipos de estressores, incluindo CPU, memória, operações de E/S e diversos outros, o que possibilita a simulação de condições extremas e controladas em ambientes de teste (KING, 2025).

Durante os testes, o Coroot registrou a reinicialização do *pod* devido à extrapolação do limite de memória configurado. As ocorrências foram documentadas e podem ser visualizadas na Figura 4.7, em um gráfico que apresenta tanto os momentos das reinicializações quanto o motivo associado, identificado como OOM Killer.

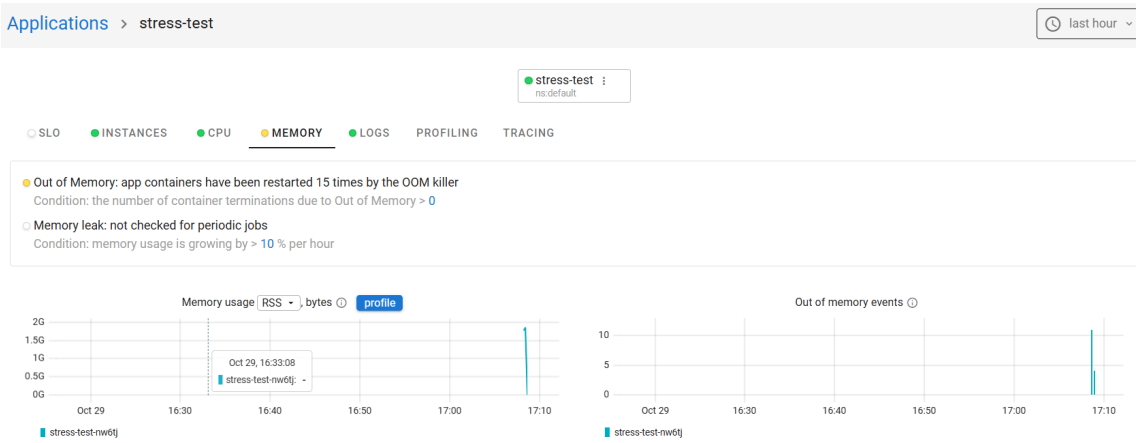

Figura 4.7: Tela sobre memória RAM

Outra analise que o coroot proporciona é em relação ao uso de CPU, que foi gerado por alto uso de CPU, mostrado na Figura 4.8.

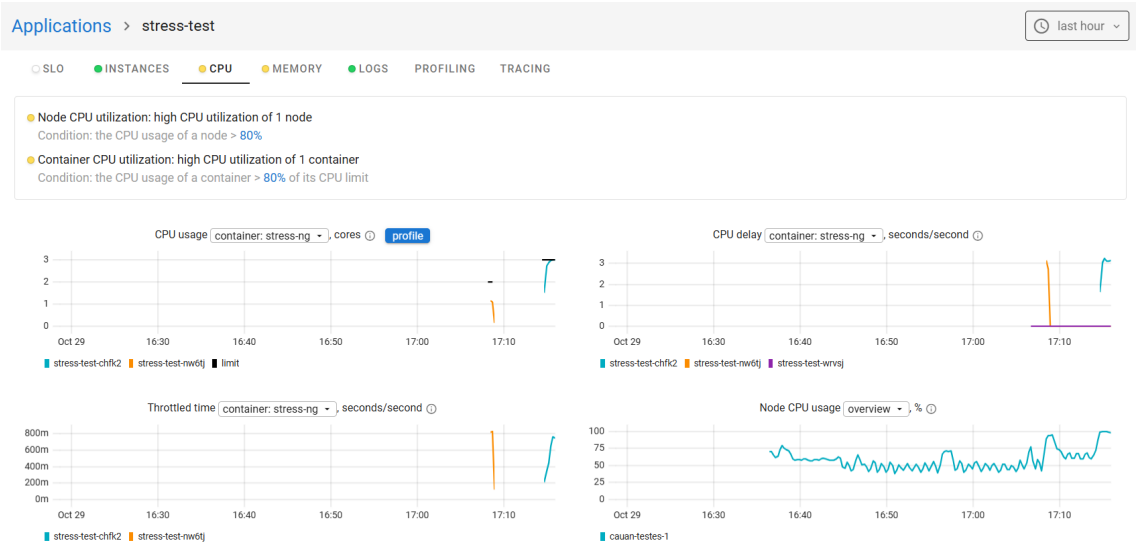

Figura 4.8: Tela sobre CPU

Sobrecarga de disco

Em sequência, foi realizado um experimento que consistiu em saturar o armazenamento da máquina virtual, preenchendo o disco com dados aleatórios até ultrapassar 90% de utilização. O objetivo foi verificar a capacidade do Netdata em detectar e sinalizar condições críticas de esgotamento de espaço em disco, validando sua eficiência no monitoramento de recursos. Como podemos ver na Figura 4.9, no Netdata foi registrada a ocorrência, concomitante com o alerta enviado pelo Discord, que está evidenciado na Figura 4.10.

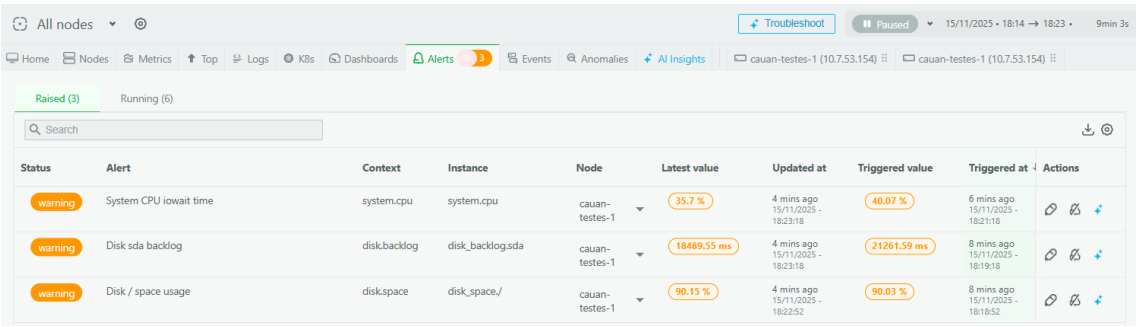

Figura 4.9: Visão geral de alertas na IDE do Netdata

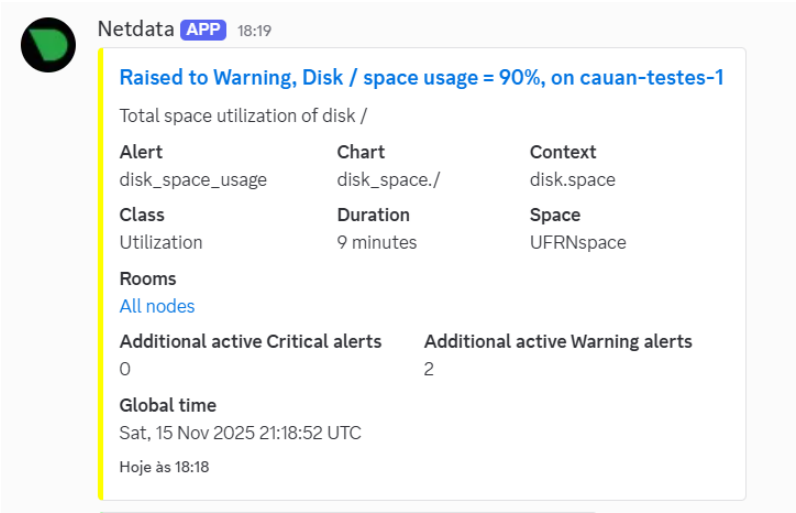

Figura 4.10: Alerta de disco do Netdata no Discord

4.3 Caso 3 - Api com alto índice de erro

A API foi desenvolvida exclusivamente para simular instabilidades e avaliar como os sistemas de observabilidade respondem a falhas e latência. Ela possui um único *endpoint* que, a cada requisição, seleciona aleatoriamente um comportamento: em 70% das vezes retorna imediatamente um erro; em 10% das vezes atrasa a resposta em 1 (um) segundo e retorna sucesso; e, nos 20% restantes, atrasa a resposta em 15 (quinze) segundos antes de responder. Múltiplas requisições foram realizadas a essa API, simulando cenários de alta latência e baixa disponibilidade, o que permitiu analisar *dashboards*, alertas e o comportamento do ambiente em condições adversas. Nesse cenário, o serviço não possui auto-instrumentação configurada, e os dados de telemetria são coletados pelo sistema eBPF do Coroot.

O painel de *tracing*, mostrado na Figura 4.11 evidencia de forma precisa o comportamento instável da API utilizada no experimento. O *HeatMap* funciona como uma visão global do sistema, permitindo observar, ao longo do tempo, a distribuição das requisições, seus tempos de resposta e seus respectivos status. Na visualização, é possível identificar

que a predominância de pontos vermelhos representa requisições que retornaram erro de forma imediata, condizente com a probabilidade de 70% de falha configurada. A intensidade do azul no HeatMap revela também a frequência de requisições: tons mais claros indicam menor volume, enquanto tons mais escuros mostram maior concentração de chamadas naquele intervalo de tempo. Essa visualização fornece uma visão sintética, porém detalhada, do comportamento do sistema, sendo fundamental para análises de desempenho e detecção de falhas, e permite identificar instantaneamente quando as anomalias começaram e como elas se distribuem, sem a necessidade de analisar *trace* individualmente.

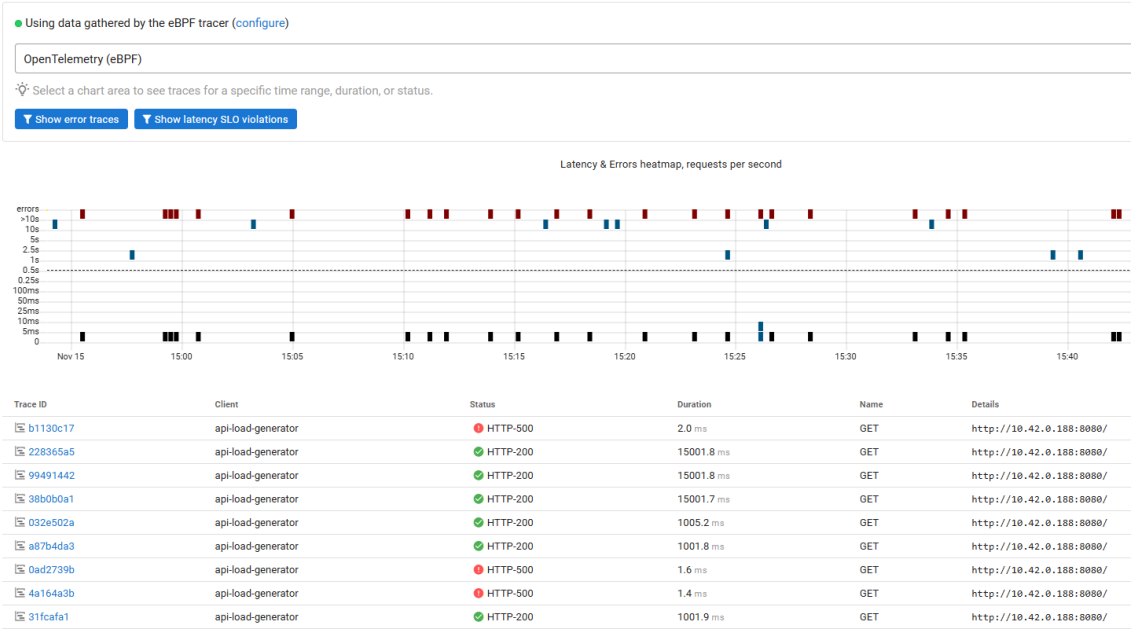

Figura 4.11: Tracing via eBPF

Podemos ver que, para este caso, o Coroot categorizou a ocorrência no painel de incidentes, evidenciado na Figura 4.12, devido à alta latência e indisponibilidade do sistema e gerou um alerta, que é evidenciado na Figura 4.13.

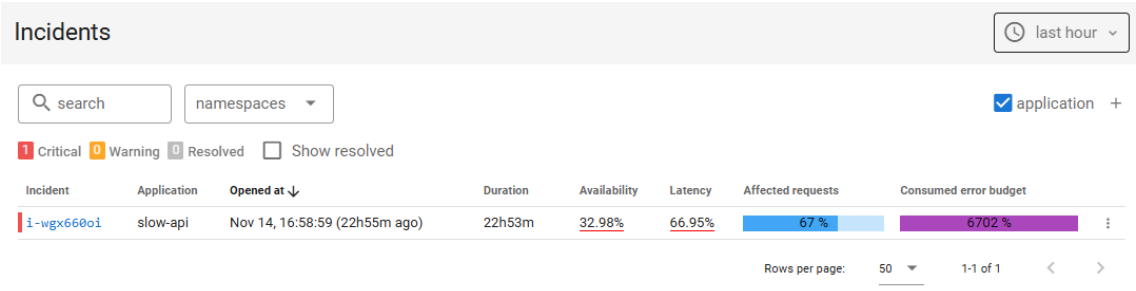

Figura 4.12: Incidente relacionado a API

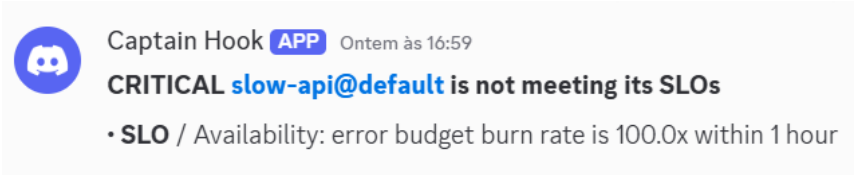

Figura 4.13: Notificação de incidente no Discord

4.4 Visão Geral

Algumas informações relevantes sobre os sistemas de monitoramento, como Coroot, Netdata e Prometheus, que não foram abordadas nos experimentos, serão apresentadas nesta seção.

4.4.1 Coroot

O Coroot é capaz de identificar todos os sistemas hospedados no *cluster* como pode ser visto na Figura 4.14, incluindo o kubelet rke2-server, responsável pelo gerenciamento dos *nodes* e pela operação do próprio cluster, proporcionando observabilidade e monitoramento abrangentes dos serviços em execução, permitindo uma visão detalhada da infraestrutura e de suas dependências.

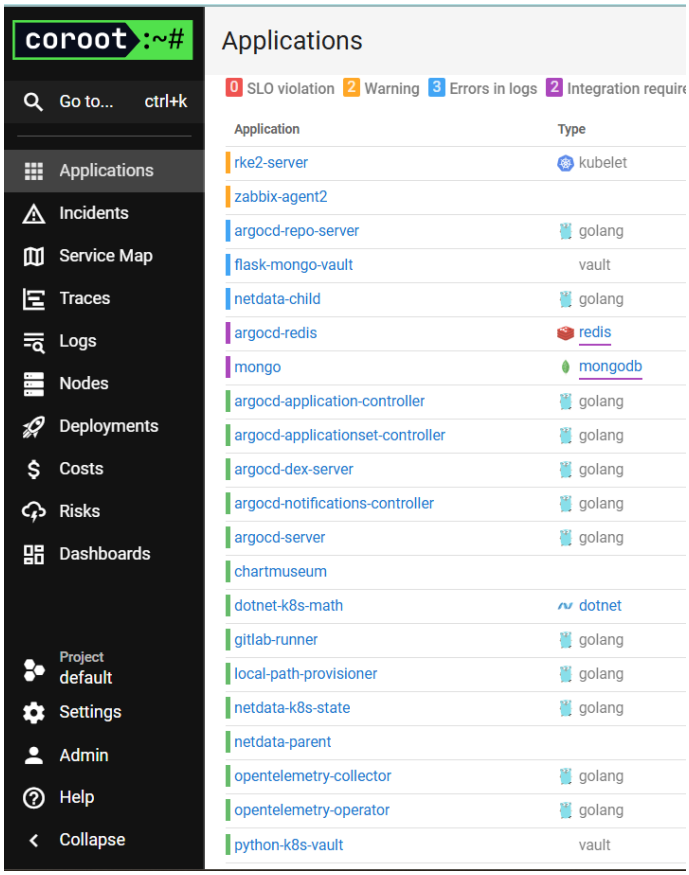

Figura 4.14: Lista de sistemas

Falhas em sistemas como bancos hospedados também são monitorados. Como pode ser visto na Figura 4.15, houve um incidente com o serviço MongoDB, um banco de dados. A partir disso, foi possível inspecionar, através de telas, informações detalhadas, como é elucidado na Figura 4.16, que contém informações de latência em requisições. Em alguns casos, ele disponibiliza painéis personalizados para o serviço, como é apresentado na Figura 4.17, onde são abordados dados sobre *queries*.

Incidents

last hour

search | namespaces | application +

0 Critical 1 Warning 0 Resolved Show resolved

Incident	Application	Opened at	Duration	Availability	Latency	Affected requests	Consumed error budget
i-nkais1i0	mongo	Nov 11, 10:48:32 (1d49m ago)	1d47m	100%	96.15%	3.8 %	385 %

Rows per page: 50 1-1 of 1

Figura 4.15: Incidente relacionado ao mongoDB

Incidents > nkais1i0

incident: nkais1i0

Incident: i-nkais1i0 Started: Nov 11, 10:48:32 (1d47m ago) Resolved: still open Duration: 1d45m Severity: WARNING Application: mongo

Service Level Objective (SLO)	Objective	Compliance
Availability	99% of requests should not fail	100%
Latency	99% of requests should be served faster than 500ms	96.15%

OVERVIEW TRACES ROOT CAUSE ANALYSIS

Latency & Errors heatmap, requests per second

0.0006/s 0.06/s 0.1/s

errors, >10s, 10s, 5s, 2.5s, 1s, 0.5s, 0.25s, 100ms, 50ms, 25ms, 10ms, 5ms, 0

Nov 11 12:00 14:00 16:00 18:00 20:00 22:00 Nov 12 02:00 04:00 06:00 08:00 10:00

Figura 4.16: Incidente com MongoDB

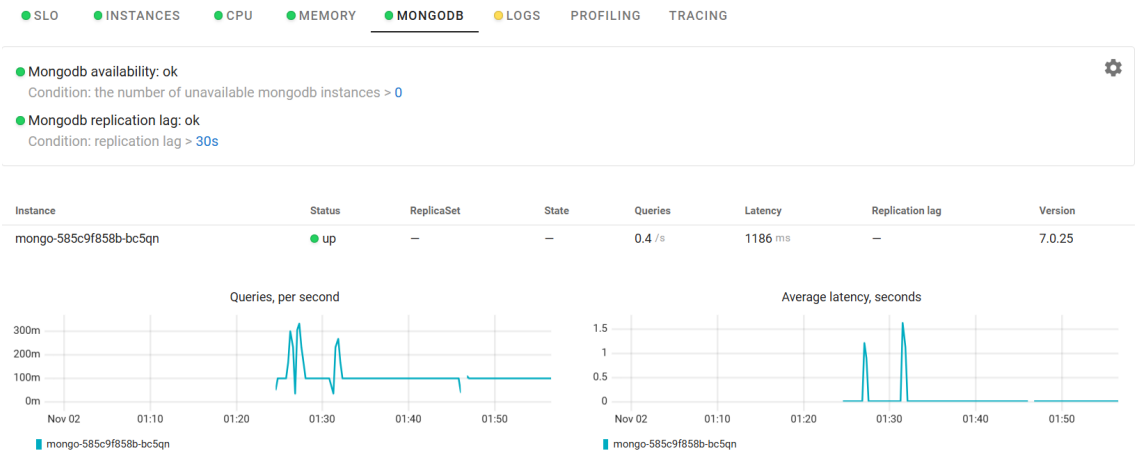

Figura 4.17: Tela de MongoDB

Monitorando o tráfego de rede, foi possível gerar um gráfico relacionando os serviços que se comunicam (Figura 4.18) e, também, a nível global, em tela especifica, como pode ser visto na Figura 4.19. Neste gráfico, é possível inspecionar as conexões que são indicadas em verde quando estão saudáveis ou em vermelho quando há falha ou alta latência, como mostrado na Figura 4.20.

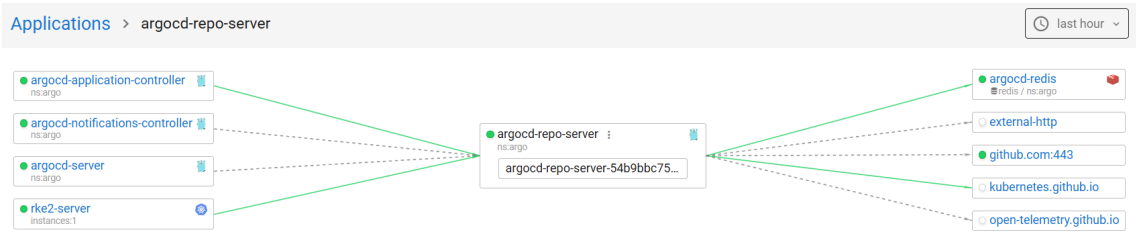

Figura 4.18: Tráfego de rede de sistema

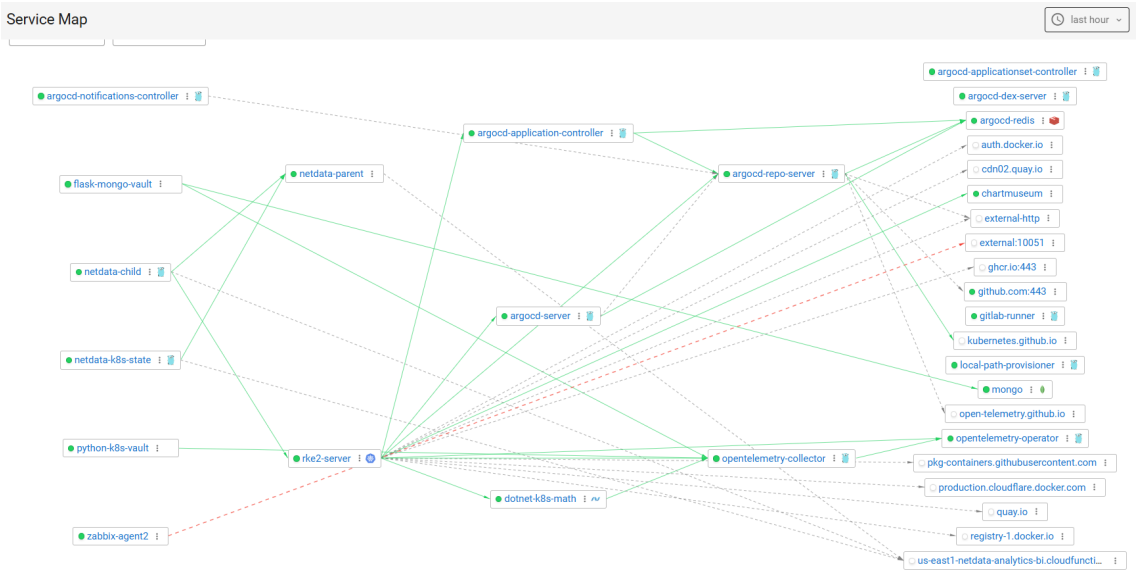

Figura 4.19: Mapa de serviços

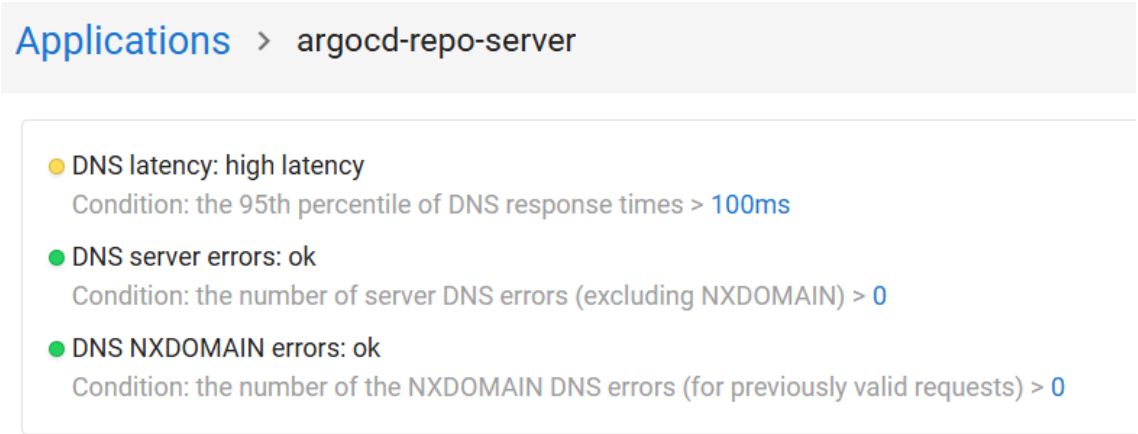

Figura 4.20: Registro de alta latência

É possível gerar uma estimativa de custo com base nos recursos gastos pelo sistema, bastando configurar o custo por CPU e por memória RAM no painel evidenciado na Figura 4.21.

Costs

last hour

The nodes are either not in a supported cloud, or the agent cannot access cloud metadata.
In this case, custom pricing is used, and you can adjust the prices per vCPU and per GB of memory. **Configure**

Nodes

Node	CPU	Memory	Price	Idle cost
cauan-testes-1 8 vCPU / 22GB	usage: request:		$262.04/mo	$152.67/mo
TOTAL			$262.04/mo	$152.67/mo

Rows per page: 10 1-1 of 1

Applications

Category	Usage costs	Allocation costs	Overprovisioning costs	Cross-AZ traffic	Internet egress traffic
application	$27.63/mo	$12.85/mo	$11.99/mo	–	–
control-plane	$8.90/mo	$15.64/mo	$12.93/mo	–	–
monitoring	$6.18/mo	$13.72/mo	$9.59/mo	–	–
TOTAL	$42.71/mo	$42.21/mo	$34.50/mo	–	–

Rows per page: 10 1-3 of 3

Figura 4.21: Tela de estimativa de custos

Fornece uma tela sobre os possíveis pontos de vulnerabilidade dos sistemas, como é mostrado na Figura 4.22. Nela, podemos ver dois riscos, sendo eles relacionados à possibilidade de perda de dados.

Risks

last hour

search namespaces application +

2 Critical 0 Warning 0 Dismissed Show dismissed

Application	Application type	Risk category	Description
argocd-redis	redis	Availability	Redis isn't replicated - data loss possible
mongo	mongodb	Availability	Mongodb isn't replicated - data loss possible

Rows per page: 50 1-2 of 2

Figura 4.22: Tela de riscos

O gráfico de *profiling*, Figura 4.23, de cada sistema hospedado mostra como a aplicação utiliza a CPU, indicando em quais funções o processamento está sendo gasto e permitindo identificar gargalos de desempenho. A partir dele, é possível inferir quais partes do código são responsáveis pela maior carga computacional, detectar *loops* ou algoritmos ineficientes. Esse tipo de análise permite localizar gargalos, entender a origem

da lentidão em *endpoints* específicos e validar como a aplicação se comporta sob carga, facilitando otimizações de código.

Com esse gráfico, podemos identificar quais funções estão consumindo mais processamento e como se distribuem entre suas subfunções. Observando a função mongo::doLoop(), percebemos que ela concentra uma parte significativa do uso de CPU, esse tempo é dividido entre suas subfunções mongo::FTDCCollectorCollection::collect(mongo::Client*), que representa 45% do processamento global, e mongo::FTDCFileWriter::writeSample[...], consumindo uma parcela menor. Dessa forma, conseguimos identificar os gargalos com alta requisição de CPU e atuar de maneira direcionada.

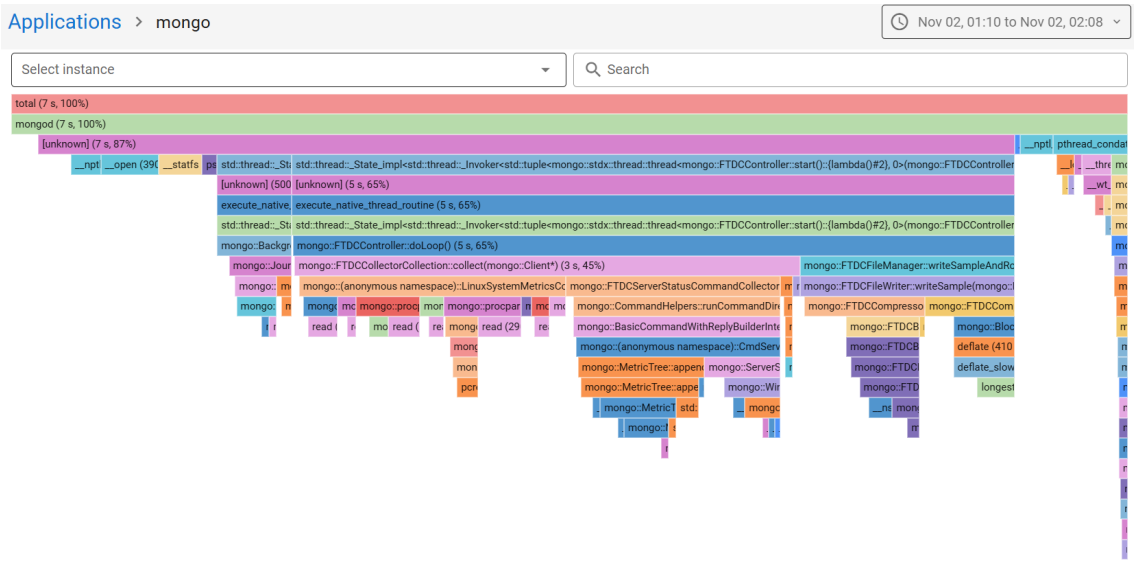

Figura 4.23: Profiling do MongoDB

4.4.2 Netdata

Como diferencial o Netdata busca monitorar os recursos e sistemas instalados no *node* que compõe o *cluster*. Trazendo dados granulares a respeito dos processos, permitindo gerar gráficos, como é mostrado na Figura 4.24, mostrando os recursos de CPU e RAM consumidos por cada processo em execução no *host*.

Figura 4.24: Tela de Inspeção de sistemas no Netdata UI

4.4.3 Prometheus

Com a *stack* do Prometheus instalada no *cluster*, dados relacionados aos *Pods* e serviços hospedados são coletados e armazenados. Com eles, foi possível visualizar os dados por meio do *dashboard* que consta na Tabela A.3, alguns de seus gráficos podem ser visualizados nas Figuras 4.25 e 4.26.

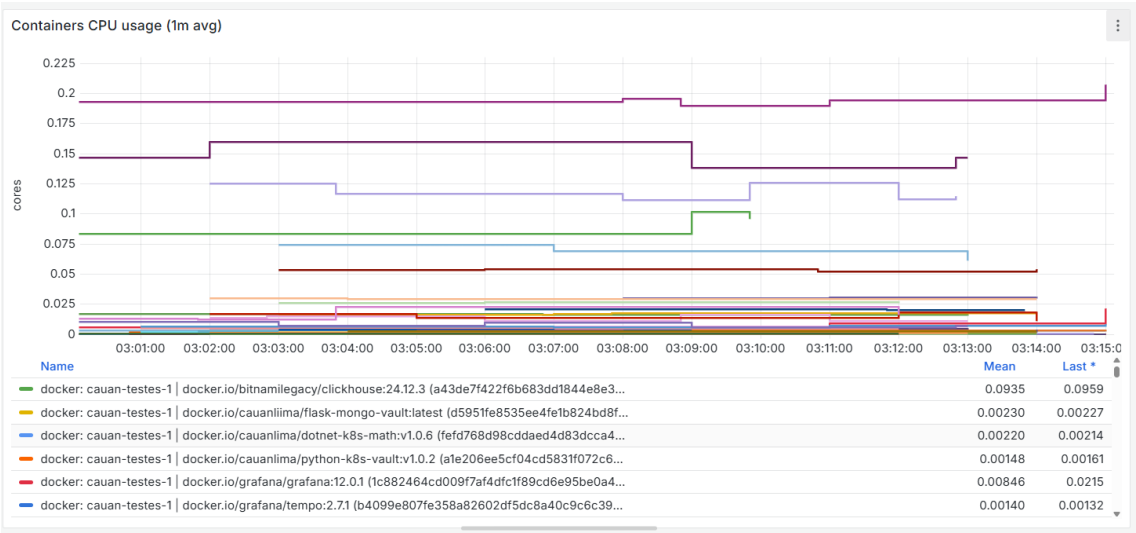

Figura 4.25: Dashboard Prometheus sobre uso memória RAM dos pods

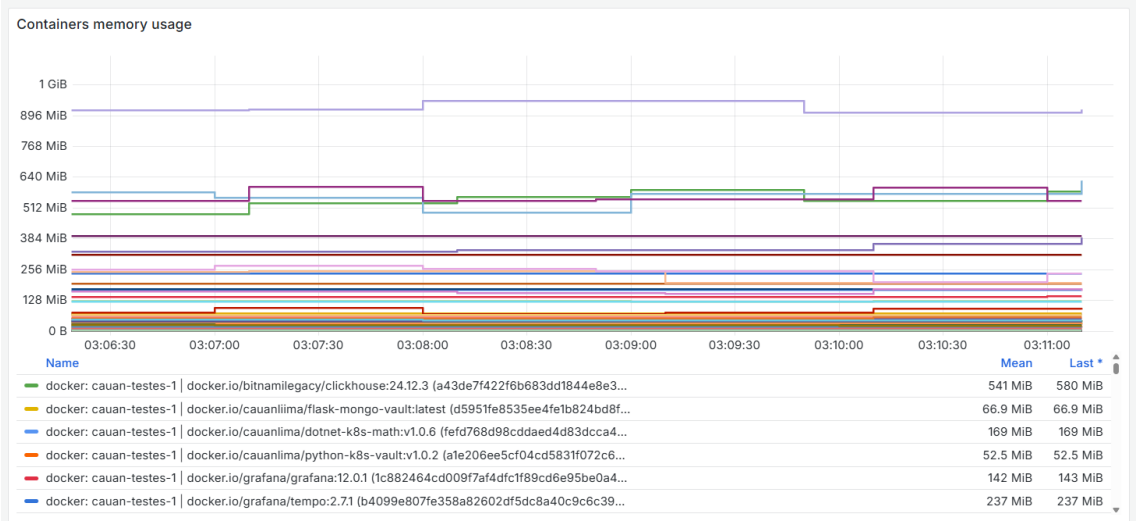

Figura 4.26: Dashboard Prometheus sobre uso de CPU dos pods

Capítulo 5

Conclusão

A partir da análise das ferramentas e práticas exploradas ao longo deste trabalho, torna-se evidente que é plenamente viável construir uma infraestrutura moderna, segura e observável. Cada ferramenta, mesmo quando apresenta funcionalidades semelhantes, oferece um diferencial importante dentro do ecossistema. O Netdata se destaca no monitoramento dos *nodes*. O Coroot fornece uma visão profunda do comportamento das aplicações no *cluster*. O OpenTelemetry possibilita rastreabilidade detalhada e geração de métricas que revelam gargalos críticos. O Prometheus atua como núcleo de monitoramento e possibilita criar sistemas de auto-gerenciamento. O HashiCorp Vault assegura uma gestão de segredos confiável e automatizada.

Apesar de possuírem funcionalidades semelhantes, o Coroot e o Netdata agregam valor ao *cluster* devido às suas diferenças, o Netdata traz melhores abordagens para o *host*, enquanto que o Coroot possui mecanismos para uma analise profunda dos serviços hospedados. Essa divergência corrobora a inclusão dos dois sistemas.

A observabilidade permite identificar pontos de latência, orientar a alocação de recursos e subsidiar decisões estratégicas, até mesmo para gestores sem conhecimento técnico aprofundado. Com isso, abre-se caminho para práticas contínuas de otimização, aumento da eficiência e evolução constante da infraestrutura. Além disso, a adoção de um ecossistema de observabilidade fortalece a cultura de operações orientadas a dados dentro da organização. À medida que métricas, *logs* e traces passam a subsidiar decisões diárias,

as equipes evoluem para um modelo mais analítico, previsível e colaborativo. Esse tipo de maturidade operacional reduz o tempo de detecção e resolução de incidentes, amplia a eficiência e confiabilidade dos serviços e contribui diretamente para o consumidor final. Assim, observabilidade deixa de ser apenas uma ferramenta técnica e se consolida como um pilar estratégico para sustentar ambientes distribuídos.

Boas práticas também estiveram presentes, como a utilização de IaC no *deploy* de serviços com o ArgoCD, o que proporciona rastreabilidade das mudanças e auditoria das configurações aplicadas aos sistemas. O versionamento de *containers* e *charts* em repositórios traz rastreabilidade das atualizações e segurança da propriedade intelectual. Por fim, a gestão de *secrets* com o Vault garante que apenas profissionais devidamente autorizados tenham acesso às informações sensíveis dos serviços.

Em síntese, a adoção de um conjunto completo de ferramentas de observabilidade e monitoramento reforça que a modernização da infraestrutura não depende exclusivamente de soluções proprietárias ou de serviços em nuvem. É possível criar um ambiente robusto, resiliente e autogerenciável, capaz de atender às demandas das aplicações modernas e de elevar de forma significativa a experiência do usuário final.

5.1 Trabalhos Futuros

Mencionado na Fundamentação Teórica, a implementação do Harbor pode proporcionar controle de versão de *containers*, segurança e análise de *containers* quando associado a ferramentas que buscam por vulnerabilidades. Para trabalhos futuros, busca-se implementar o Harbor como componente da infraestrutura.

Outro trabalho relevante que pode ser realizado é comparar o Netdata com o Zabbix, uma ferramenta que também tem como objetivo o monitoramento de VMs, possibilitando a aquisição de métricas que podem ser visualizadas no Grafana.

A auto-instrumentação é muito útil por eliminar a necessidade de alterar o código

dos sistemas, principalmente em situações em que não é possível ter acesso ao código-fonte. Entretanto, possui algumas diferenças em relação à instrumentação via código. Um trabalho relevante é fazer um comparativo entre as formas de instrumentação, trazendo vantagens, desvantagens e benefícios de ambas as formas.

Por fim, busca-se compor uma arquitetura Kubernetes que apresente a maior contingência de mecanismos e ferramentas, tanto para monitoramento quanto para observabilidade, visando nortear o progresso contínuo dos sistemas hospedados.

Referências Bibliográficas

ARTIFACT. *Artifact Hub Documentation*. 2025. Acesso em: 10 nov. 2025. Disponível em: <https://artifacthub.io/docs/>.

CHARTMUSEUM. *ChartMuseum Documentation*. 2025. Acesso em: 16 nov. 2025. Disponível em: <https://chartmuseum.com/>.

CNCF. *Argo CD Documentation*. 2025. Acesso em: 16 nov. 2025. Disponível em: <https://argo-cd.readthedocs.io/>.

CNCF. *Harbor*. 2025. Acesso em: 16 nov. 2025. Disponível em: <https://goharbor.io/>.

COROOT. *Coroot — Observabilidade Unificada com Métricas, Logs e Rastreamentos*. 2025. Acesso em: 29 out. 2025. Disponível em: <https://coroot.com/>.

DOCKER. *Docker Hub*. 2025. Acesso em: 16 nov. 2025. Disponível em: <https://docs.docker.com/docker-hub/>.

GRAFANA. *Grafana Documentation*. 2025. Acesso em: 16 nov. 2025. Disponível em: <https://grafana.com/>.

GRAFANA. *Grafana Tempo Documentation*. 2025. Acesso em: 16 nov. 2025. Disponível em: <https://grafana.com/oss/tempo/>.

HASHICORP. *HashiCorp Vault Documentation*. 2025. Acesso em: 16 nov. 2025. Disponível em: <https://developer.hashicorp.com/vault>.

HELM. *Helm Documentation*. 2025. Acesso em: 10 nov. 2025. Disponível em: <https://helm.sh/docs/>.

KING, C. I. *stress-ng: A Tool to Load and Stress a Computer System*. 2025. Acesso em: 16 nov. 2025. Disponível em: <https://github.com/ColinIanKing/stress-ng>.

KUBERNETES. *Kubernetes Documentation*. 2025. Acesso em: 29 out. 2025. Disponível em: <https://kubernetes.io/docs/home/>.

MAJORS, C.; FONG-JONES, L.; KUELBS, G. *Observability Engineering: Achieving Production Excellence*. [S.l.]: O'Reilly Media, 2022. ISBN 1492076449; 9781492076445.

MART, O. et al. Observability in kubernetes cluster: Automatic anomalies detection using prometheus. In: *2020 IEEE 22nd International Conference on High Performance Computing and Communications (HPCC)*. [S.l.]: IEEE, 2020. p. 789–798.

NETDATA. *Coroot vs Netdata Comparison*. 2025. Acesso em: 2025. Disponível em: <https://www.netdata.cloud/comparisons/coroot/>.

NETDATA. *Netdata*. 2025. Acesso em: 29 out. 2025. Disponível em: <https://www.netdata.cloud/>.

OPENTELEMETRY. *OpenTelemetry*. 2025. Acesso em: 29 out. 2025. Disponível em: <https://opentelemetry.io/>.

OPENTELEMETRY. *OpenTelemetry Collector*. 2025. Acesso em: 29 out. 2025. Disponível em: <https://opentelemetry.io/docs/collector/>.

OPENTELEMETRY. *OpenTelemetry Collector Contrib*. 2025. Acesso em: 29 out. 2025. Disponível em: <https://github.com/open-telemetry/opentelemetry-collector-contrib>.

OPENTELEMETRY. *OpenTelemetry Operator Automatic Instrumentation*. 2025. Acesso em: 17 nov. 2025. Disponível em: <https://opentelemetry.io/docs/platforms/kubernetes/operator/automatic>.

OPENTELEMETRY. *OpenTelemetry Operator for Kubernetes*. 2025. Acesso em: 29 out. 2025. Disponível em: <https://opentelemetry.io/docs/platforms/kubernetes/operator/>.

PORTWORX. *The Voice of Kubernetes Experts Report 2024*. 2024. Acesso em: 24 nov. 2025. Disponível em: <https://portworx.com/resources/voice-of-kubernetes-expert-report>.

PROMETHEUS. *Prometheus Documentation: Overview*. 2025. Acesso em: 16 nov. 2025. Disponível em: <https://prometheus.io/docs/introduction/overview/>.

VAULT. *Injecting Vault Secrets into Kubernetes Pods via a Sidecar*. 2019. Acesso em: 10 nov. 2025. Disponível em: <https://www.hashicorp.com/pt/blog/injecting-vault-secrets-into-kubernetes-pods-via-a-sidecar>.

Apêndice A

Informações adicionais

O Apêndice reúne informações adicionais produzidas ao longo da implementação e avaliação do ambiente estudado. A Tabela A.1 apresenta a finalidade das principais ferramentas hospedadas no *cluster*, oferecendo uma visão geral de seus papéis e contribuições para o funcionamento da arquitetura. A Tabela A.2 registra alguns dos incidentes identificáveis por meio do Coroot. A Tabela A.3 reúne os *dashboards* do Grafana. A Tabela A.4 apresenta os *charts* empregados no *deploy* dos serviços instalados no *cluster*, acompanhados de seus respectivos repositórios. Por fim, a Tabela A.5 oferece uma comparação entre as soluções Coroot e Netdata, evidenciando suas principais diferenças. As Figuras A.1 e A.2 complementam esse conjunto de materiais ao ilustrar, respectivamente, os manifestos ArgoCD utilizados para o *deploy* do sistema Calculadora e do serviço Vault, evidenciando, neste último caso, o uso de arquivos *values* para a customização do *chart* Helm.

Tabela A.1: Utilidade das ferramentas

Ferramenta	**Principais Funcionalidades**
HashiCorp Vault	Armazena segredos com segurança. Composto pelo Vault Server, onde os segredos são armazenados, e pelo Vault Injector, capaz de injetar o vault init e o vault sidecar em aplicações no Kubernetes.
OpenTelemetry Operator	Gerencia automaticamente agentes e coletores no Kubernetes, injeta sidecars de telemetria, aplica configurações e facilita instrumentação sem alterar código.
OpenTelemetry Collector	Recebe, processa e exporta métricas, logs e traces para diversos backends como Prometheus, Grafana, Tempo e OpenSearch.
Prometheus	Coleta e armazena métricas; realiza *scraping*, mantém uma base temporal e possibilita alertas via Alertmanager.
Grafana Tempo	Armazena traces, compatível com OpenTelemetry.
Coroot	Monitoramento e observabilidade de sistemas no cluster; coleta métricas via eBPF, útil para aplicações sem instrumentação OpenTelemetry; identifica latências, dependências e gargalos.
Netdata	Monitoramento de recursos dos nodes como CPU, memória, disco e processos; fornece análise em tempo real, detecção de anomalias e alertas.
Harbor	Registry seguro para imagens; permite armazenar, gerenciar, assinar e escanear imagens para vulnerabilidades, replicação e políticas de retenção.
ArgoCD	Ferramenta GitOps para Kubernetes; sincroniza o estado dos serviços com projetos Git, faz deploy, rollback e visualização dos recursos.
Grafana	Plataforma de visualização e dashboards; integra com Prometheus, Grafana Tempo e outros; permite alertas e *dashboards* dinâmicos.
ChartMuseum	Repositório de charts Helm; permite armazenar, versionar e distribuir charts, suportando operações como upload, download e indexação automática.
cert-manager	Automatiza a emissão e renovação de certificados TLS no Kubernetes.

Tabela A.2: Principais incidentes detectados pelo Coroot

Categoria	Incidente	Descrição
Recursos	OOM Kill (Out of Memory)	Processo encerrado por ultrapassar o limite de memória disponível.
	CPU Throttling	Execução limitada pelo cgroup devido ao consumo excessivo de CPU.
	High CPU Usage	Uso de CPU consistentemente elevado.
	High Memory Usage	Memória próxima ao limite configurado.
	Memory Leak	Crescimento contínuo do consumo de memória, indicando ausência de liberação adequada.
	Disk Pressure / I/O Wait	Operações de disco lentas, com elevado tempo de espera.
	Pod Evicted	Pod removido do nó por insuficiência de recursos.
Rede	High Network Latency	Atraso significativo na comunicação entre serviços.
	Connection Timeouts	Expiração de conexões ou falhas de estabelecimento TCP.
	Packet Drops / Retransmits	Perda ou retransmissão de pacotes, impactando a estabilidade da rede.
Armazenamento	High Disk Usage	Volume ou filesystem próximo da capacidade máxima.
	Disk I/O Saturation	Operações de leitura/gravação lentas devido à saturação do disco.
Aplicação	High Error Rate (5xx)	Taxa elevada de erros HTTP retornados pela aplicação.
	Slow Response Time	Respostas acima do tempo aceitável configurado.
	Unhealthy Pod	Falhas nas verificações de readiness ou liveness.
	Restart Loop	Ciclo contínuo de reinicializações (CrashLoopBackOff).
	Dependency Failures	Indisponibilidade ou falhas em serviços externos dependidos.
Infraestrutura	Node NotReady / Unreachable	Nó inacessível, indisponível ou sem envio de métricas.
	Agent Missing Data	Agente Coroot sem dados recentes ou desconectado.
	Prometheus / Exporter Down	Backend de métricas ou exporters inacessíveis.

Tabela A.3: Dashboards do Grafana utilizados para monitoramento

ID	Título	Rev.	URL
6417	Kubernetes cluster monitoring (via Prometheus)	1	https://grafana.com/grafana/dashboards/315-kubernetes-cluster-monitoring-via-prometheus/
19419	OpenTelemetry APM	6	https://grafana.com/grafana/dashboards/19419-opentelemetry-apm/

Tabela A.4: Serviços Instalados no Cluster via Helm Charts

Repositório	**Chart**	**Versão**
https://chartmuseum.github.io/charts	chartmuseum	3.10.3
https://charts.jetstack.io	cert-manager	v1.18.0
https://coroot.github.io/helm-charts	coroot	0.20.3
https://grafana.github.io/helm-charts	tempo	1.21.0
https://grafana.github.io/helm-charts	grafana	9.2.3
https://netdata.github.io/helmchart	netdata	3.7.130
https://open-telemetry.github.io/opentelemetry-helm-charts	opentelemetry-collector	0.126.0
https://open-telemetry.github.io/opentelemetry-helm-charts	opentelemetry-operator	0.90.3
https://prometheus-community.github.io/helm-charts	prometheus	27.8.0
https://helm.releases.hashicorp.com	vault	0.28.1

Tabela A.5: Comparativo entre Coroot e Netdata

Feature	**Coroot**	**Netdata**
Data Granularity	Varies	1-second
Primary Data Collection	eBPF (Zero-instrumentation)	Lightweight Agent
Setup & Discovery	Automatic via eBPF	Zero-configuration auto-discovery
AI/ML Capabilities	AI-powered RCA (Enterprise only)	Unsupervised ML on all metrics at the edge
Open Source Option	✓	✓
Kubernetes Monitoring	✓	✓
VM & Bare Metal Monitoring	✓	✓
Real-time Dashboards	✓	✓
Pricing Model	Community (Free) & Enterprise	Transparent per-node pricing
Self-Hosted Option	✓	✓

Fonte: Adaptado de Netdata – Comparação Coroot vs Netdata (NETDATA, 2025a).

Figura A.1: Manifesto do sistema dotnet-k8s-math

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dotnet-k8s-math
  namespace: argo
spec:
  project: default
  source:
    repoURL: http://chartmuseum.chartmuseum:8080
    chart: dotnet-k8s-math-chart
    targetRevision: 0.1.0
  destination:
    server: https://kubernetes.default.svc
    namespace: dotnet-k8s-math
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Figura A.2: Manifesto do serviço Vault

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: vault
  namespace: argo
spec:
  project: default
  source:
    repoURL: https://helm.releases.hashicorp.com
    chart: vault
    targetRevision: 0.28.1
    helm:
      valueFiles: []
      values: |
        server:
          service:
            type: NodePort
            nodePort: 30280
          dev:
            enabled: true
  destination:
    server: https://kubernetes.default.svc
    namespace: vault
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```